



**COMSATS University Islamabad,
Sahiwal Campus.**

Law Sphere

By

Muhammad Asswad Tahir

CIIT/SP22-BCS-068/SWL

Muhammad Jamshaid

CIIT/SP22-BCS-080/SWL

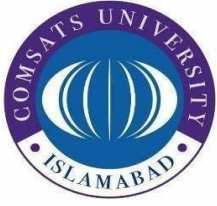
Muhammad Zain

CIIT/SP22-BCS-097/SWL

Supervisor

Mr. Fayez Afzaal

Bachelor of Science in Computer Science (2022-2026)



COMSATS University Islamabad, Pakistan

Law Sphere

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Muhammad Asswad Tahir

CIIT/SP22-BCS-068/SWL

Muhammad Jamshaid

CIIT/SP22-BCS-080/SWL

Muhammad Zain

CIIT/SP22-BCS-097/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Asswad Tahir

Muhammad Jamshaid

Muhammad Zain

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “**Law Sphere**” was developed by **Aswad Tahir (CIIT/SP22-BCS-068/SWL)**, **Muhammad Jamshaid (CIIT/SP22-BCS-080/SWL)** and **Muhammad Zain (CIIT/SP22-BCS-097/SWL)** under the supervision of Mr. Fayeز Afzaal and co-supervisor Dr. Javed Ferzund that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mr. Fayeز Afzaal

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Co-Supervisor

Dr. Javed Ferzund

Assistant Professor

Department of Software Engineering

COMSATS University Islamabad, Sahiwal Campus

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

The Law Sphere is a web-based legal management platform designed to streamline case handling, client communication, appointment scheduling, and administrative operations for legal practitioners and law firms. Developed with a strong focus on accessibility, automation, and data security, the system bridges the gap between lawyers and clients by offering a centralized and intuitive environment for managing legal services.

The platform enables clients to register, log in, book appointments, view case updates, upload documents, and communicate with their assigned lawyer through a user-friendly interface. Lawyers can efficiently manage case records, monitor ongoing proceedings, store confidential documents, track appointment schedules, and oversee client activity through a dedicated dashboard. This dual-interface structure ensures seamless interaction between clients and lawyers while maintaining high standards of data accuracy, confidentiality, and operational transparency.

A key innovation of the Law Sphere is its smart case-recommendation and document-analysis support, which assists lawyers in organizing information, retrieving case files, and managing workflow more effectively. Automated reminders and notifications enhance time management by alerting lawyers and clients about upcoming appointments, required documents, and important case deadlines. The system uses secure JWT-based authentication to protect user access, while the backend—powered by Node.js and MongoDB—ensures reliability, scalability, and fast data processing for sensitive legal operations.

By integrating automation, secure data handling, and smart digital tools, the Lawyer Management System delivers an all-in-one solution for modern legal management. It reduces manual workload, improves communication efficiency, and enhances decision-making by providing lawyers and clients with a structured, transparent, and intelligent platform. Overall, the system establishes a robust digital foundation for law firms looking to modernize operations and offer improved legal services in today's technology-driven world.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor of “Mr. Fayez Afzaal” and co-supervisor “Dr. Javed Ferzund”. Without their personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are deeply thankful for their encouragement and continual support during this work.

We also extend our heartfelt gratitude to our families and friends who have been a constant source of support and motivation throughout this journey.

Aswad Tahir

Muhammad Jamshaid

Muhammad Zain

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
NoSQL	Non-Structured Query Language
REST	Representational State Transfer
UI	User Interface

Table of Contents

1 Introduction.....	1
1.1 Relevance to Course Modules	1
1.2 Project Background.....	2
1.3 Literature Review.....	3
1.4 Analysis from Literature Review	4
1.5 Methodology and Software Lifecycle.....	4
1.5.1 Rationale behind Selected Methodology:	5
2 Problem Definition.....	7
2.1 Problem Statement	7
2.2 Deliverables and Development Requirements	8
2.2.1 Major Deliverables	8
2.2.2 Development Requirements:	10
2.3 Current System.....	11
3 Requirement Analysis.....	13
3.1 Use Case Diagram.....	13
3.2 Use Case Description	14
3.3 Functional Requirements	24
3.3.1 User Registration and Authentication (UC7)	24
3.3.2 Search and Filter Lawyers/Services	25
3.3.3 View Lawyer Profile Details (UC5).....	25
3.3.4 Share Lawyer Profile Details	25
3.3.5 Schedule and Verify Appointment (UC1, UC9)	25
3.3.6 Get AI Lawyer/Case Recommendations	25
3.3.7 Manage Lawyer Profiles and Credentials (UC10)	26
3.3.9 Add and Approve New Lawyers (UC6, UC10)	26
3.3.10 Edit and Update Lawyer Profile (UC6).....	26
3.3.11 Manage Appointments and Disputes (UC12)	26
3.3.12 Manage Dashboard (UC11, UC13).....	27

3.4 Non-Functional Requirements	27
3.4.1 Performance Requirements	27
3.4.2 Scalability Requirements.....	27
3.4.3 Security Requirements	27
3.4.4 Usability Requirements	28
3.4.5 Reliability Requirements	28
3.4.6 Maintainability Requirements	28
3.4.7 Portability Requirements.....	28
3.4.8 Data Integrity Requirements	28
3.4.9 Availability Requirements.....	29
3.4.10 Extensibility Requirements	29
3.4.11 Backup and Recovery Requirements	29
3.4.12 Compliance and Legal Requirements	29
3.4.14 Load Handling Requirements.....	29
4 Design and Architecture	31
4.1 System Architecture	31
31	
4.2 Data Representation	32
4.2.1 Key Entities and Relationships	33
4.2.2 ER Diagram.....	34
4.3 Process Flow/Representation	35
4.3.1 Activity Diagrams	35
4.3.1.1 Client Activities:	35
4.3.1.2 Administrator Activities:.....	35
4.3.1.8 Admin Manage Dashboard Activity Diagram	41
4.4 Design Models	42
4.4.1 Sequence Diagram.....	42
4.4.2 Class Diagram	47
5 Implementation	50
5.1 Algorithm	50

5.1.1 User Authentication.....	50
5.1.2 Schedule Appointment	50
5.1.3 Payment Processing.....	51
5.1.4 Feedback Submission.....	51
5.1.5 Live Chat Management	52
5.1.6 Admin Verification & Approval	52
5.1.8 Dashboard Analytics	53
5.1.10 Admin Management.....	53
5.2 External APIs	54
5.3 User Interface.....	55
5.3.1 Lawyer Account Creation Screen.....	56
5.3.2 Home Screen	57
5.3.3 About Us Screen.....	58
5.3.4 Contact Us Screen	59
5.3.5 Admin Dashboard Screen.....	60
5.3.6 Case Management Screen	61
5.3.7 User Dashboard Screen	62
6 Testing and Evaluation	64
6.1 Manual Testing	64
6.1.1 System Testing	65
6.1.2 Unit Testing.....	66
6.1.3 Functional Testing.....	68
6.1.4 Integration Testing	70
6.2 Tools and Technologies	71
6.3 Summary	73
7 Conclusion	75
7.1 Future Work.....	76
7.1.1 Mobile Application Development.....	76
7.1.2 AI-Based Legal Assistance and Automation	76

7.1.3 Real-Time Chat and Virtual Legal Assistant	76
7.1.4 Secure Online Payment Integration.....	76
7.1.5 Advanced Data Analytics Dashboard.....	76
7.1.6 Cloud Deployment and Scalability	76
7.1.7 Enhanced Security and Compliance.....	76
7.1.8 Integration With Court Systems & Legal Databases	77
8 References.....	79

List of Tables

Table 1.1.1: Relevance to course modules.....	2
Table 3.2.1: Schedule appointments	14
Table 3.2.2: Provide feedback	15
Table 3.2.3: Make payments	15
Table 3.2.4: Live chatting	16
Table 3.2.5: View record	17
Table 3.2.6: Manage profiles	18
Table 3.2.7: Login/Signup	18
Table 3.2.8: Payment verification	19
Table 3.2.9: Appointment verification.....	20
Table 3.2.10: Approve lawyers profile	21
Table 3.2.11: Analysis and reporting.....	22
Table 3.2.12: Financial dispute handling.....	22
Table 3.2.13: Monitor system logs	23
Figure 4.1: Law Sphere System architecture	31
Figure 4.3: Law Sphere ER Diagram.....	34
Figure 4.4: User Login and Registration Activity Diagram	36
Figure 4.4: Schedule Appointment Activity Diagram	37
Figure 4.5: Make Payment Activity Diagram.....	38
Figure 4.6: Provide Feedback Activity Diagram	39
Figure 4.7: Live Chat Activity Diagram	40
Figure 4.8: Manage Dashboard Activity Diagram.....	41
Figure 4.9: Sequence Diagram for User Registration and Login.....	42

Figure 4.10: Sequence Diagram for Provide Feedback	43
Figure 4.11: Book Appointment Sequence Diagram	44
Figure 4.12: Sequence Diagram for Admin Approve Lawyer Profile	45
Figure 4.13: Admin Manage Dashboard Sequence Diagram	46
Figure 4.14: Class Diagram	47
Table 15.2.1: External APIs.....	54
Figure 5.1: Lawyer Account Creation Screen.....	56
Figure 5.2: Home Screen	57
Figure 5.3: About Us Screen.....	58
Figure 5.5: AI Legal Recommendation Hub Screen	59
Figure 5.5: Contact Us Screen	59
Figure 5.6: Admin Dashboard Screen.....	60
Figure 5.7: Case Management Screen	61
Figure 5.8: User Dashboard Screen	62
Table 6.1.1.1: System Testing Table	65
Table 16.1.2.1: Unit Function Table	67
Table 6.1.3.1: Test Case Table.....	69
Table 6.1.4.1: Integration Testing Table.....	70
Table 6.2.1: Tools and Technologies Table.....	71

List of Figures

Figure 3.1: Law sphere use case	13
Figure 4.12: Sequence diagram for admin approve lawyer profile	45
Figure 4.13: Admin manage dashboard sequence diagram	46
Figure 4.14: Class diagram	47
Figure 5.1: Lawyer account creation screen	56
Figure 5.6: Admin dashboard screen	60
Figure 5.7: Case management screen.....	61
Figure 5.8: User dashboard screen.....	62

Chapter 1

INTRODUCTION

1 Introduction

The rapid advancement of digital technologies has redefined operational models across numerous sectors, and the legal services domain is no exception. Clients now demand accessible, reliable, and streamlined solutions that simplify their interactions with legal professionals. Traditional methods of contacting lawyers, scheduling appointments, maintaining legal records, and resolving financial disputes often involve inefficient manual procedures that lead to delays, miscommunication, and administrative burdens. These limitations make it challenging for clients, lawyers, and administrators to engage in an organized and transparent workflow.

Law Sphere has been developed to address these challenges by providing an integrated, web-based legal management platform that modernizes and centralizes interactions between clients, lawyers, and system administrators. The platform facilitates appointment scheduling, payment processing, live chat communication, feedback provision, and profile management within a unified ecosystem. On the lawyer's side, the system supports profile verification, appointment validation, payment review, and access to client records. The administrative module oversees high-level operations such as approving lawyer profiles, handling financial disputes, monitoring system logs, conducting data analysis, and managing system-wide activities in real time.

Built using contemporary full-stack web technologies, Law Sphere ensures scalability, reliability, and secure communication through robust authentication and role-based access mechanisms. The system is designed to offer an intuitive user experience that enhances trust and transparency while streamlining legal service workflows. By combining efficient software engineering methodologies with a structured architectural design, Law Sphere contributes to the digital transformation of legal service management and promotes a user-centric operational framework.

1.1 Relevance to Course Modules

The development of the **Law Sphere** platform integrates multiple concepts, theories, and technical skills acquired throughout the **Bachelor of Science in Computer Science (BSCS)** degree program. This project combines areas such as software engineering, artificial intelligence, database management, web development, and human-computer interaction to create a comprehensive and intelligent legal service management system. Each course module contributes to different aspects of the system as outlined below:

Table 1.1.1: Relevance to course modules

Course Module	Application in Law Sphere
Software Engineering	Software Engineering SRS & SDD documents, UML diagrams
Database system	Designed and implemented data storage and retrieval mechanisms using MongoDB for user, case insights / legal analytics .
Web Technologies	Developed responsive web interfaces using React and Tailwind CSS, ensuring seamless interaction and usability.
Artificial Intelligence	Incorporated AI and ML models for legal service management system.
Data Science	Utilized data visualization and statistical analysis to extract actionable insights from lawyer recommendations.
Information Security	Implemented JWT-based authentication, secure data transmission, and rate limiting to prevent unauthorized access.

Table 1.1.1: Relevance to course module outlines the project not only reinforces theoretical understanding but also demonstrates practical implementation of interdisciplinary knowledge, aligning with the academic objectives of the Computer Science curriculum. By integrating these subjects, the project provides a comprehensive demonstration of real-world application of the theoretical learning gained over eight semesters.

1.2 Project Background

The legal services industry has traditionally relied on manual procedures for appointments, client communication, document management, and payment handling. With the growing need for digital transformation, there is rising demand for solutions that offer efficiency, accuracy, and centralized management. Numerous legal service portals exist, but many remain limited to basic contact information or simple appointment forms, lacking the robust functionality needed to support continuous interaction between clients, lawyers, and administrators.

Law Sphere was conceptualized to bridge this gap by introducing a comprehensive platform that manages legal service workflows in an organized, interactive, and transparent manner. The system is designed to support clients in scheduling appointments, making secure payments, reviewing previous records, providing feedback, and communicating directly with lawyers. Lawyers benefit from an efficient verification mechanism, appointment review capabilities, profile management, and access to client-related data. Administrators oversee the entire platform through modules that handle profile approvals, disputes, analytics, and system log monitoring.

Developed as a modular and scalable system, Law Sphere reflects best practices in full-stack development and user experience design. It provides a structured approach to digitalizing legal operations and demonstrates how modern technologies can streamline communication, improve accountability, and enhance service delivery in the legal domain.

1.3 Literature Review

Digital transformation within the legal industry has seen significant attention over recent years, particularly with the rise of online appointment platforms, client-attorney communication portals, and automated management tools. Studies such as “Technological Adoption in Legal Practice Management” (Khan & Roberts, 2020) emphasize the necessity of digital systems that can support scheduling, communication, and secure data storage. Platforms such as LawPay, Clio, and MyCase offer foundational tools but often lack deeper modules that integrate multi-role access and administrative oversight within a single unified environment.

Research by Ahmed and Lee (2021) in “Enhancing Client-Lawyer Interactions through Web-Based Systems” highlights the importance of transparency, accessibility, and structured communication tools within legal service platforms. Their study indicates that client satisfaction improves significantly when direct communication channels, feedback systems, and appointment management tools are integrated into a single interface.

Other works, such as “Secure Web Architecture for Legal Data Management” (Javed & Malik, 2022), focus on the need for confidentiality and data protection, as legal systems often handle sensitive personal information. These studies reinforce that a well-designed legal platform must incorporate strong security mechanisms, role-based access control, efficient data handling, and a user-friendly environment.

The existing literature collectively reveals that current platforms rarely combine client services, lawyer functionalities, and administrator controls in a holistic system. This highlights the need for a comprehensive solution like Law Sphere.

1.4 Analysis from Literature Review

The literature indicates that although numerous digital solutions have been introduced in the legal industry, most platforms offer only partial automation of legal service workflows. Many systems prioritize simple appointment booking or document storage without integrating real-time communication, feedback mechanisms, or administrative oversight. These limitations hinder the effectiveness of digital transformation within the sector.

Law Sphere emerges as a response to these shortcomings, offering a cohesive structure that integrates client, lawyer, and administrator workflows into a unified system. The inclusion of functionalities such as appointment scheduling, payment handling, profile verification, dispute management, and system log monitoring demonstrates a comprehensive approach to digital legal management. The integration of role-based access control enhances security, while the modular architecture addresses scalability and long-term adaptability.

By aligning with gaps identified in existing research, Law Sphere represents a modern and complete solution capable of improving trust, communication, and efficiency in legal service interactions.

1.5 Methodology and Software Lifecycle

The development of Law Sphere follows the Agile Software Development Lifecycle (SDLC), allowing continuous enhancement through iterative development and user-centered design. Agile methodology supports evolving requirements, which is particularly essential in a system that integrates multiple user roles and diverse functionalities. Throughout the project, feedback loops were incorporated to refine interface design, backend logic, and security mechanisms.

The process began with requirement gathering, where functional and non-functional requirements of clients, lawyers, and administrators were identified. This was followed by system design, during which architectural models, workflow diagrams, and database schemas were created to outline system structure and interactions. Implementation involved developing the user interfaces and backend services while ensuring seamless integration with authentication modules and communication components.

Testing played a crucial role, ensuring reliability through unit testing, integration testing, and scenarios reflecting real user interactions. Deployment considerations involved optimizing the system for accessibility, performance, and maintainability. The maintenance phase includes updating the platform, enhancing security, improving usability, and scaling modules based on future needs.

1.5.1 Rationale behind Selected Methodology:

Agile methodology was selected due to its adaptability, incremental delivery model, and strong emphasis on user collaboration. Unlike rigid development approaches such as the Waterfall model, Agile allows continuous refinement of requirements and accommodates evolving system needs. This flexibility is essential for a project like Law Sphere, where client modules, lawyer tools, and administrative functionalities must align smoothly and adapt to practical usage patterns.

The iterative nature of Agile ensures that each module—such as appointment management, profile verification, or analytics—can be developed, tested, and improved independently. This reduces risk, improves development efficiency, and ensures the final system is robust, user-friendly, and aligned with stakeholder expectations.

Chapter 2

PROBLEM DEFINITION

2 Problem Definition

The legal services sector is undergoing a steady transition toward digitalization, yet many existing systems still depend on manual procedures, fragmented communication channels, and inefficient workflows. Clients often struggle to locate suitable lawyers, schedule appointments, maintain legal records, or manage payments in an organized manner. Lawyers face difficulties in verifying appointments, reviewing payments, handling client information, and communicating effectively within a secure environment. Administrators experience challenges in monitoring disputes, verifying lawyer profiles, reviewing system logs, and analyzing operational data due to the absence of centralized and automated tools. These limitations collectively lead to delays, miscommunication, and reduced service quality.

Law Sphere was conceptualized to overcome these issues by introducing a unified legal service management platform that centralizes client–lawyer interactions and administrative oversight. The system provides structured modules for secure user authentication, appointment scheduling, profile management, payment handling, feedback generation, chat communication, and administrative monitoring. This chapter defines the underlying problem, outlines the system deliverables, and highlights the gaps in current legal service platforms that Law Sphere aims to address.

2.1 Problem Statement

Traditional legal service platforms offer limited functionality, often providing only basic lawyer listings and static contact details. These systems lack dynamic mechanisms for scheduling appointments, recording interactions, managing payments, or verifying professional profiles. Clients frequently encounter inconvenience due to the absence of centralized communication channels and proper record-keeping, resulting in inefficient service access. Lawyers face difficulty maintaining appointment schedules, reviewing payments, and accessing client histories due to fragmented tools and manual workflows.

Further examination shows that existing solutions lack structured communication modules, feedback-processing mechanisms, and administrative oversight. There is typically no system to manage disputes related to payments or service delivery, and administrators are unable to track system-wide activities or access historical logs. The lack of an integrated, role-based system leads to operational inefficiencies for all stakeholders.

Given these limitations, the legal industry requires a modern, digitally managed system capable of automating core operations, improving communication, securing sensitive data, and establishing transparency in service delivery. Law Sphere aims to satisfy these needs by combining structured workflows, real-time interaction modules, and centralized administrative monitoring into a single cohesive platform.

2.2 Deliverables and Development Requirements

Law Sphere aims to deliver a complete, fully functional digital platform that manages interactions among clients, lawyers, and administrators while ensuring security, usability, and streamlined communication. The following subsections outline the major system deliverables and the corresponding development requirements essential for building and deploying the platform successfully.

2.2.1 Major Deliverables

Project deliverables for LawSphere include both functional and technical outputs designed to improve access to legal services and enhance trust between clients and legal professionals. Functional deliverables focus on user-facing features such as secure user registration and login, lawyer verification, client case management, appointment scheduling, online consultation, secure payment processing, and client review and feedback mechanisms. Each functional component addresses key shortcomings in traditional legal service systems, particularly the lack of transparency, accessibility, and efficient case handling.

The lawyer verification module ensures that only licensed and authorized legal professionals are registered on the platform, strengthening credibility and user trust. The client profile and case management module enables systematic documentation of case details, legal documents, consultation history, and deadlines, reducing the risk of missed hearings or incomplete records. This structured approach enhances case tracking and improves communication between lawyers and clients.

The appointment and consultation scheduling module streamlines the booking process for both physical and virtual meetings, increasing accessibility to legal services for users in remote or underserved regions. Integrated online payment and billing functionality minimizes administrative overhead and ensures secure, transparent financial transactions. Additionally,

the client review and rating system promotes accountability among legal professionals and supports continuous service improvement.

Technical deliverables support these functional modules by providing a robust and scalable infrastructure. LawSphere is developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) to ensure high performance, flexibility, and maintainability. React.js delivers a responsive and intuitive user interface, while Node.js and Express.js handle backend logic and secure API management. MongoDB provides efficient data storage for user profiles, case records, and transaction data. JWT-based authentication and role-based access control are implemented to protect sensitive legal information and ensure secure system access.

An admin dashboard serves as a centralized control panel for platform management, including user verification, dispute resolution, content moderation, and analytical reporting.

Comprehensive system documentation, including architecture design, API documentation, testing reports, and user manuals, ensures long-term maintainability and supports future system enhancements.

Development requirements define the resources necessary to implement these deliverables effectively. Hardware requirements include development workstations and testing devices with stable internet connectivity. Software requirements consist of Node.js, MongoDB, React.js, Express.js, code editors such as Visual Studio Code, API testing tools, and version control systems such as Git and GitHub. Human resources include frontend and backend developers, UI/UX designers, quality assurance testers, and project supervisors, each contributing to the development of a secure, user-friendly, and reliable legal web platform.

By clearly defining these deliverables and requirements, LawSphere ensures that the development team is equipped with the necessary tools, technologies, and expertise to build an efficient, trustworthy, and scalable digital legal services platform.

2.2.1.1 Web-Based Legal Management Platform

The core deliverable of Law Sphere is a complete web-based platform that integrates all major functionalities required for managing client–lawyer interactions. This platform provides a unified system where users can access legal services without relying on manual communication methods or external tools.

2.2.1.2 Client Service Module

The client environment includes all essential features for scheduling appointments, submitting secure payments, maintaining personal profiles, accessing previous legal records, and communicating directly with lawyers through an integrated chat system. These functionalities ensure that clients can manage their legal engagements efficiently and transparently.

2.2.1.3 Lawyer Service Module

Lawyers are provided with dedicated functionalities that support appointment verification, payment review, profile management, and access to client feedback or previous case interactions. This module centralizes the lawyer's operational tasks, reducing manual workload and enabling more organized case handling.

2.2.1.4 Administrative Dashboard

A comprehensive administrative dashboard serves as one of the major outcomes of the project. Administrators are equipped with tools to verify lawyer profiles, resolve financial disputes, review activity logs, and analyze system performance. This centralized administrative environment ensures smooth platform governance and operational oversight.

2.2.1.5 Deployment and Documentation

The final deliverables also include the fully developed and deployed web application along with complete technical documentation, source code, and deployment configurations. These components ensure system maintainability, transparency, and long-term support.

2.2.2 Development Requirements:

2.2.2.1 Frontend Development Environment

The frontend of Law Sphere requires a responsive and intuitive interface capable of supporting the workflows of clients, lawyers, and administrators. The interaction design must ensure usability across devices while maintaining a consistent structure for forms, dashboards, and communication modules.

2.2.2.2 Backend Development Environment

The backend serves as the core processing unit of the platform. It is responsible for handling business logic, processing user requests, securing communication, and managing all system

operations. A stable and scalable backend ensures uninterrupted functionality and accurate transaction processing.

2.2.2.3 Database Management Environment

An efficient database system is required to store user profiles, appointment records, payment information, feedback details, and system logs. Structured data storage ensures that information can be retrieved quickly and securely while supporting the platform's multi-role architecture.

2.2.2.4 Development and Deployment Toolchain

The project also depends on a complete set of development tools for coding, debugging, version control, testing, and deployment. These tools allow the system to be built in a systematic manner while maintaining reliability, performance, and accessibility across different operating environments.

2.3 Current System

Most existing digital platforms for legal services offer minimal functionality, primarily focusing on basic contact details or simple appointment request forms. These systems operate with static interfaces and lack automated workflows that support interactive communication between clients and lawyers. Key operational components such as appointment verification, payment review, feedback evaluation, and dispute handling remain manual, consuming time and reducing service efficiency. There is no centralized mechanism for storing legal records, viewing past interactions, or providing personalized services to users.

Furthermore, current platforms seldom include an administrative environment capable of monitoring activities, verifying lawyer credentials, analyzing system statistics, or reviewing logs for security and performance. Due to limited scalability and insufficient security mechanisms, existing systems often fail to protect sensitive user information effectively. These shortcomings create gaps in reliability, transparency, and operational flow.

Law Sphere addresses these issues by introducing a multi-role, integrated architecture that supports real-time communication, centralized appointment scheduling, secure payment management, profile verification, feedback processing, and administrative monitoring.

Chapter 3

REQUIREMENT ANALYSIS

3 Requirement Analysis

Requirement Analysis, or Requirement Engineering, is the process of defining user expectations for new software to be built or modified. In software engineering, it is sometimes referred to loosely by names such as requirements gathering or requirements capturing. It involves gathering, analyzing, documenting, validating, and managing the needs and conditions required to meet stakeholder expectations, ensuring alignment with project goals.

3.1 Use Case Diagram

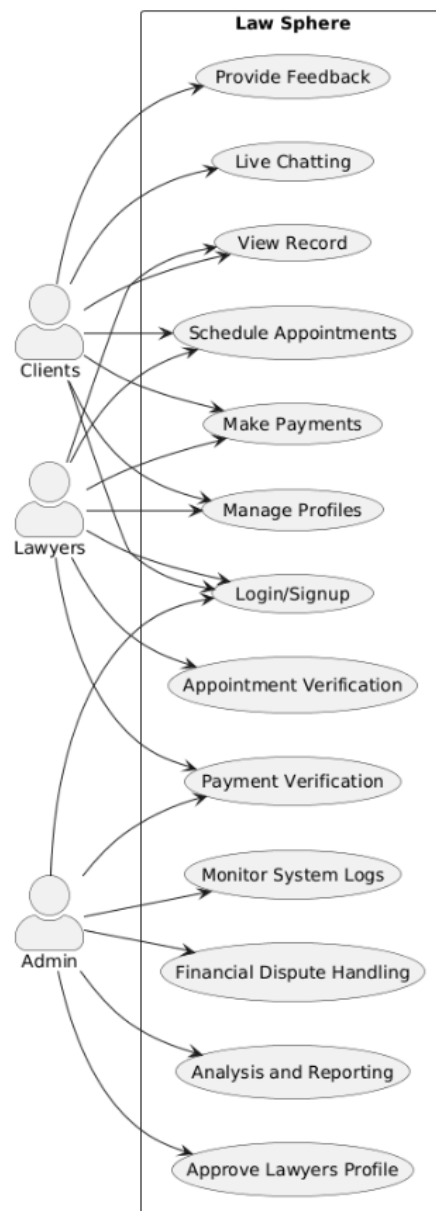


Figure 3.1: Law Sphere Use Case

3.2 Use Case Description

Table 3.2.1: Schedule appointments

Use case id	UC #01
Use case name	Schedule Appointments
Actors	Clients, Lawyers
Description	Enables a Client to search for a Lawyer, check their availability, and submit a request to book a consultation time slot.
Trigger	Client clicks the "Book Appointment" button on a Lawyer's profile.
Pre-condition	Client is logged in (UC7). Lawyer has an approved profile (UC10) and available time slots.
Post-condition	A pending appointment request is created, and the Lawyer is notified for verification (UC9).
Normal flow	1. Client searches/selects a Lawyer. 2. Client views Lawyer's calendar and selects a date/time. 3. Client provides a brief case summary. 4. Client clicks "Submit Request". 5. System creates a pending appointment record.
Exceptions	Booking fails due to a concurrent booking attempt or a system error.
Business rules	Appointment time slots are unique and cannot be double-booked. Lawyers must set a minimum lead time (e.g., 4 hours) for new bookings.
Alternative flow	If the selected slot is no longer available, the system suggests the next three closest available slots.

Table 3.2.2: Provide feedback

Use case id	UC#02
Use case name	Provide Feedback
Actors	Clients
Description	Allows Clients to submit ratings and written reviews for the Lawyer after a consultation is marked as completed.
Trigger	The appointment status is marked as 'Completed'.
Pre-condition	Client is logged in (UC7) and has completed an appointment with the Lawyer.
Post-condition	The feedback (rating and review) is successfully saved to the Lawyer's profile and updates the Lawyer's average rating.
Normal flow	1. Client receives a prompt to rate the Lawyer. 2. Client enters a star rating (1-5) and optional review text. 3. Client clicks "Submit Feedback". 4. System validates and saves the feedback record.
Alternative flow	• Client can choose to skip the rating and submit only the review text, or vice versa.
Exceptions	Client attempts to submit feedback for the same appointment more than once.
Business rules	Only one feedback submission is allowed per completed appointment. Feedback must be submitted within 30 days of the appointment completion date.

Table 3.2.3: Make payments

Use case id	UC#03
Use case name	Make Payments
Actors	Clients, Lawyers
Description	Allows Clients to pay consultation fees and enables Lawyers to initiate the withdrawal of their verified earnings.

Trigger	Client: Confirmed appointment is due. Lawyer: Lawyer requests a payout from their earnings balance.
Pre-condition	Client: Confirmed and scheduled appointment. Lawyer: Verified earnings are available for withdrawal
Post-condition	Transaction is successfully initiated and recorded; for payouts, the request is sent for verification (UC8).
Normal flow	1. User (Client/Lawyer) navigates to the Payments/Earnings section. 2. Client: Selects payment method and enters details. 3. Lawyer: Enters withdrawal amount and confirms. 4. System processes the transaction via the payment gateway.
Alternative flow	The payment gateway declines the transaction, and the user is prompted to try a different method.
Exceptions	System fails to communicate with the payment gateway, resulting in a transaction error.
Business rules	All client payments are subject to a platform transaction fee. Lawyer payouts must meet a minimum withdrawal threshold (e.g., \$50).

Table 3.2.4: Live chatting

Use case id	UC#04
Use case name	Live Chatting
Actors	Clients
Description	Provides a secure, real-time text communication channel between a Client and their confirmed Lawyer.
Trigger	Client opens the chat window associated with an active or recently completed appointment.
Pre-condition	Client is logged in (UC7). An appointment must be confirmed (UC9) or recently completed (within a set timeframe).
Post-condition	Messages are successfully exchanged, and the conversation history is securely logged.

Normal flow	1. Client navigates to the relevant appointment. 2. Clicks the "Chat" icon. 3. Types and sends a message. 4. Lawyer receives the message and responds in real-time.
Alternative flow	If the Lawyer is offline, the message is stored and delivered as an asynchronous notification when they next log in.
Exceptions	A network failure prevents real-time message delivery.
Business rules	Chat communication must be related to the scheduled appointment/case. All chat logs are subject to a 5-year retention policy for legal and audit purposes.

Table 3.2.5: View record

Use case id	UC#05
Use case name	View Record
Actors	Clients, Lawyers
Description	Allows Clients to view their complete history (appointments, payments, feedback) and Lawyers to view their case history and earnings reports.
Trigger	User clicks "History," "Records," or "Reports" from their main dashboard.
Pre-condition	User is logged in (UC7).
Post-condition	The requested historical data is retrieved from the database and displayed securely.
Normal flow	1. User navigates to the records section. 2. User selects a filter (e.g., date range, status). 3. System queries the database and displays the filtered list of records.
Alternative flow	User selects an option to export the displayed records as a downloadable CSV or PDF file.

Exceptions	Data retrieval fails due to a database connection timeout or error.
Business rules	Users are strictly forbidden from viewing records associated with other users or cases outside their direct involvement.

Table 3.2.6: Manage profiles

Use case id	UC#06
Use case name	Manage Profiles
Actors	Clients, Lawyers
Description	Enables Clients and Lawyers to update their personal information, contact details, and, for Lawyers, their professional credentials and pricing.
Trigger	User clicks the "Edit Profile" or "Settings" option.
Pre-condition	User is logged in (UC7).
Post-condition	The user's profile information is updated and stored in the database.
Normal flow	1. User navigates to the profile management page. 2. Edits required fields (e.g., address, profile picture). 3. Clicks "Save Changes". 4. System validates and commits the changes.
Alternative flow (Lawyer)	If a Lawyer updates their credentials, the profile status changes to 'Pending Re-approval' and requires Admin review (UC10).
Exceptions	Input data fails validation (e.g., entering letters in a phone number field).
Business rules	Lawyer credential updates automatically trigger the Admin approval process (UC10). The Lawyer's public profile cannot be edited while a review is pending.

Table 3.2.7: Login/Signup

Use case id	UC#07
--------------------	-------

Use case name	Login/Signup
Actors	Clients, Lawyers, Admin
Description	Allows all user types to register a new account or authenticate their identity to access the platform.
Trigger	User opens the application or selects the "Login/Register" option.
Pre-condition	User is not currently logged in.
Post-condition	User is successfully authenticated, and the appropriate user-specific dashboard is loaded.
Normal flow	1. User selects "Login". 2. User enters Email/Username and Password. 3. System verifies credentials against the database. 4. User is granted access to the system.
Alternative flow	User selects "Sign Up" and is guided through the initial profile creation process (UC6).
Exceptions	System fails to authenticate due to incorrect credentials or an account being suspended/locked.
Business rules	Passwords must meet minimum complexity requirements (e.g., 8 characters, one number, one special character). Email verification is mandatory for all new accounts.

Table 3.2.8: Payment verification

Use case id	UC#08
Use case name	Payment Verification
Actors	Lawyers, Admin
Description	The mandatory process to confirm the successful transfer of funds (Client payments confirmed by Lawyer; Lawyer payouts approved by Admin).
Trigger	A payment (UC3) is reported as complete by the payment gateway, or a payout request is initiated by a Lawyer.

Pre-condition	A financial transaction exists in a pending or unverified state.
Post-condition	The transaction status is updated to 'Verified' or 'Approved', and the system's financial ledgers are finalized.
Normal flow (Admin)	1. Admin views pending payout requests. 2. Admin verifies external bank transfer confirmation. 3. Admin clicks "Approve Payout". 4. System updates the record status.
Normal flow (Lawyer)	1. Lawyer is notified of an incoming Client payment. 2. Lawyer confirms the payment reflects in their earnings balance.
Alternative flow	Admin detects a mismatch in records and initiates Financial Dispute Handling (UC12).
Exceptions	The Admin or Lawyer attempts to verify a transaction that has already been verified or canceled.

Table 3.2.9: Appointment verification

Use case id	UC#09
Use case name	Appointment Verification
Actors	Lawyers
Description	Allows a Lawyer to review the details of a Client's request (UC1) and officially accept or reject the booking.
Trigger	Lawyer receives a notification for a new pending appointment request.
Pre-condition	Lawyer is logged in (UC7) and has a pending appointment request.
Post-condition	Appointment status is updated to 'Confirmed' or 'Rejected', and the Client receives an automated notification.
Normal flow	1. Lawyer views the pending appointment details (case summary, time). 2. Lawyer clicks either "Accept" or "Reject". 3. System updates the status in the calendar and notifies the Client.

Alternative flow	If the Lawyer rejects the request, they can optionally propose up to two alternative time slots for the Client.
Exceptions	System fails to update the status due to a database lock or error.
Business rules	Lawyers must respond to a pending request (Accept/Reject) within 24 hours, or the request automatically expires and is canceled.

Table 3.2.10: Approve lawyers profile

Use case id	UC#10
Use case name	Approve Lawyers Profile
Actors	Admin
Description	Enables the Admin to review a newly registered Lawyer's professional credentials and officially activate their profile for public view.
Trigger	A Lawyer completes their profile setup (UC6) and submits it for initial platform approval.
Pre-condition	Admin is logged in (UC7) and there is a pending Lawyer profile in the approval queue.
Post-condition	The Lawyer's profile status is updated to 'Approved' (active) or 'Rejected' (inactive).
Normal flow	1. Admin navigates to the 'Profile Approvals' queue. 2. Admin reviews documents and details. 3. Admin clicks "Approve" or "Reject" with a mandatory reason for rejection. 4. System updates the status and notifies the Lawyer.
Alternative flow	Admin flags the profile and requests the Lawyer to upload clearer/additional documentation.
Exceptions	The system automatically suspends the review process if the Lawyer's submitted documents are expired.
Business rules	Profiles require Admin approval before being publicly searchable. All licensing documentation must be dated as current and verified against a third-party source.

Table 3.2.11: Analysis and reporting

Use case id	UC#11
Use case name	Analysis and Reporting
Actors	Admin
Description	Allows the Admin to generate analytical reports, monitor platform key performance indicators (KPIs), and review service metrics.
Trigger	Admin selects a report type (e.g., Financial Summary, Lawyer Performance) from the dashboard.
Pre-condition	Admin is logged in (UC7).
Post-condition	A report, typically with data visualizations and key summaries, is generated and displayed.
Normal flow	• 1. Admin selects the report type. • 2. Admin defines filters (e.g., date range, geographical area). • 3. Admin clicks "Generate Report". • 4. System processes data and renders the requested visualization.
Alternative flow	Admin schedules a recurring report to be delivered to a designated email address weekly or monthly.
Exceptions	Report generation fails due to the complexity of the query or timeout on massive datasets.
Business rules	Client identifying information (PII) must be anonymized in all aggregate reports. Access to reports is strictly limited to the Admin role.

Table 3.2.12: Financial dispute handling

Use case id	UC#12
Use case name	Financial Dispute Handling
Actors	Admin
Description	Provides the Admin with the necessary interface and tools to investigate, mediate, and resolve formal financial conflicts (e.g., unauthorized charges, refund requests).

Trigger	A Client or Lawyer submits a formal dispute claim related to a payment or payout.
Pre- condition	Admin is logged in (UC7) and a financial dispute is pending investigation.
Post-condition	The dispute is marked as 'Resolved', and any necessary financial adjustments (refunds, fees) are processed.
Normal flow	• 1. Admin views the dispute claim details. • 2. Admin reviews logs (UC13) and transaction history (UC8). • 3. Admin rules on the dispute (e.g., "Issue Partial Refund"). • 4. System executes the ruling and notifies both parties.
Alternative flow	Admin can request further clarification or documentation from the parties involved before making a ruling.
Exceptions	The system is unable to process a required financial adjustment due to a locked transaction.
Business rules	Dispute claims must be formally submitted within 30 days of the transaction date. The Admin's final ruling on the financial dispute is binding.

Table 3.2.13: Monitor system logs

Use case id	UC#13
Use case name	Monitor System Logs
Actors	Admin
Description	Allows the Admin to access, filter, and view system activity, security audit trails, and technical error logs for maintenance and troubleshooting.
Trigger	Admin selects the "System Logs" or "Audit Trail" option from the Admin dashboard.

Pre- Condition	Admin is logged in (UC7).
Post-Condition	The requested log entries are securely retrieved and displayed in a readable format.
Normal flow	• 1. Admin navigates to the log viewer. • 2. Admin sets a filter (e.g., log severity, user ID, date range). • 3. System loads the filtered log entries. • 4. Admin analyzes the data for errors or security issues.
Alternative flow	Admin can download a complete log archive for offline analysis.
Exceptions	The log viewer fails to load or refresh due to a heavy load on the logging service.
Business rules	All critical security events (logins, profile changes) must be logged with a timestamp and user ID. Logs must be retained for a minimum of 90 days.

3.3 Functional Requirements

Functional requirements define the specific behaviors, operations, and services that the **Law Sphere Lawyer Management Platform** must perform to fulfill its intended objectives. These requirements describe how the system should behave under specific conditions and how **Clients, Lawyers, and Administrators** will interact with the platform to accomplish their tasks (UC1 - UC13). The following subsections outline the major functionalities of the Law Sphere system.

3.3.1 User Registration and Authentication (UC7)

The system must provide a secure mechanism for Clients, Lawyers, and Administrators to register, log in, and manage authentication sessions. Users should be able to create accounts using their email addresses and passwords. The system must validate credentials during login and ensure session management using secure tokens (JWT). Incorrect login attempts should trigger appropriate error messages, and the system should prevent unauthorized access by enforcing password complexity and input validation rules.

3.3.2 Search and Filter Lawyers/Services

The platform must allow Clients to search for available Lawyers based on filters such as legal specialization (e.g., family law, criminal defense), location, fee range, and professional rating. The search feature should provide real-time results that match the applied criteria. Additionally, the system must optimize search speed to display relevant lawyer profiles efficiently and include sorting options (e.g., lowest fee, highest rating, newest join date).

3.3.3 View Lawyer Profile Details (UC5)

Clients should be able to view complete details of each Lawyer profile. This includes professional experience, specializations, hourly/consultation fees, images (if provided), office map location, and client reviews (UC2). The system must ensure that the data retrieved is accurate, up-to-date, and displayed in an organized and visually appealing format. It should also manage errors gracefully if a profile is unavailable or removed (e.g., due to Admin suspension).

3.3.4 Share Lawyer Profile Details

The system should provide Clients with the ability to share Lawyer profiles via direct links or professional networking platforms. Each shared profile should generate a unique, trackable link. In case of failure to connect to external social APIs, the system must offer an alternative copy-link feature to ensure that the sharing process remains uninterrupted.

3.3.5 Schedule and Verify Appointment (UC1, UC9)

The system must allow Clients to request and Lawyers to confirm appointments for consultations. The appointment module should include date and time selection, conflict checking against the Lawyer's schedule, and email confirmations. If the chosen time slot is already booked, the system must display alternative available options. Appointment data must be stored securely in the database, and both Client and Lawyer dashboards should reflect real-time booking status (Pending, Confirmed, Canceled).

3.3.6 Get AI Lawyer/Case Recommendations

The system must incorporate artificial intelligence to recommend Lawyers or related legal services to Clients based on their search history, stated legal needs, and behavior. These recommendations should be dynamically generated using machine learning models integrated with the platform's database. If the AI module is temporarily unavailable, the system should display cached or fallback results to maintain functionality.

3.3.7 Manage Lawyer Profiles and Credentials (UC10)

The Administrator must have full control over all Lawyer profiles and credentials within the system. This includes the ability to view, review, and manage existing profile data. The system must allow Admins to perform bulk actions such as enabling, disabling, or highlighting verified profiles. Any modification to the approval status (UC10) should be immediately reflected on the user interface.

3.3.8 Analyze Platform and Financial Trends (UC11)

The system must enable Administrators to access AI-driven analytics that reflect real-time platform trends, fee fluctuations, and user demand for specific legal services. The AI module should process data from multiple sources to generate detailed graphs and charts, helping the Admin make informed business decisions regarding marketing and Lawyer capacity.

3.3.9 Add and Approve New Lawyers (UC6, UC10)

Lawyers should be able to add their complete profile information, including specialization, experience, fee structure, and supporting credential documents. The system must validate all required fields before submission to ensure data integrity. Uploaded documents should be stored securely via integrated cloud services. Admin approval (UC10) is mandatory before the Lawyer profile becomes active on the platform.

3.3.10 Edit and Update Lawyer Profile (UC6)

The system must provide Lawyers with the ability to edit and update existing profile details when necessary. This includes modifications to fees, availability status, or contact information. All changes should be reflected instantly on the frontend and stored in the database. Data validation and change logs should be maintained, and critical updates (like credentials) may trigger a re-approval process by the Admin.

3.3.11 Manage Appointments and Disputes (UC12)

Administrators must be able to view all booked appointments and manage them effectively. This includes approving, modifying, or canceling appointments for administrative reasons. Furthermore, the Admin must be able to handle financial disputes (UC12), ensuring synchronization between the Client and Lawyer records so that updates are visible immediately to both parties.

3.3.12 Manage Dashboard (UC11, UC13)

The Admin dashboard must serve as a centralized hub providing insights into the system's overall performance (UC11). It should display real-time analytics on total users, active Lawyers, verified appointments, and revenue metrics. Additionally, the dashboard should automatically refresh at fixed intervals and allow filtering of analytics data for detailed monitoring and log review (UC13)

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the **Law Sphere Lawyer Management Platform** must meet to ensure a reliable, secure, and efficient user experience. These requirements describe how the system performs its functions rather than what it does. They focus on aspects like security, usability, performance, maintainability, and scalability that contribute to the system's overall quality and long-term sustainability.

3.4.1 Performance Requirements

The Law Sphere platform must provide fast response times and ensure that lawyer search, AI recommendations, and dashboard analytics load efficiently. Under normal load conditions, the system should respond within 2–3 seconds for user requests. The backend APIs and database queries must be optimized to handle concurrent requests smoothly, and caching mechanisms should be implemented to improve response times for frequently accessed data, especially for Lawyer availability checks.

3.4.2 Scalability Requirements

The system architecture must be scalable to accommodate future growth in the number of users, Lawyer registrations, and appointment transactions. The backend infrastructure should support horizontal scaling, allowing additional servers or databases to be added seamlessly. The frontend, developed with modern frameworks, should efficiently handle large data sets of Lawyer profiles and case records without performance degradation.

3.4.3 Security Requirements

Security is a critical aspect of the Law Sphere platform, given the sensitive nature of legal data. The system must use JWT (JSON Web Tokens) for user authentication and role-based authorization to ensure that sensitive data is accessible only to authorized Clients, Lawyers, or Admins. All communication between client and server must occur over HTTPS to prevent data

interception. Passwords should be encrypted using hashing algorithms like bcrypt, and input validation must be enforced to prevent common web attacks.

3.4.4 Usability Requirements

The platform's user interface must be intuitive, responsive, and easy to navigate for users of all backgrounds. The design should follow WCAG 2.1 accessibility standards, ensuring usability for differently-abled users. Elements such as search filters, appointment booking forms, and chat interfaces must be clearly labeled and logically structured. Both the Client, Lawyer, and Admin dashboards should provide smooth interaction and clear feedback for every action performed.

3.4.5 Reliability Requirements

The Law Sphere system must remain stable and functional even during high traffic or minor network disruptions. It should provide 99% uptime, ensuring continuous availability for Lawyer searches, appointment scheduling, and payment processing. Automatic retries and error-handling mechanisms should be implemented to recover from transient failures without user disruption.

3.4.6 Maintainability Requirements

The software must be modular and maintainable, allowing easy updates and integration of new features. Each module (Client, Lawyer, Admin, Appointment, Payment, AI) should be developed independently to simplify debugging and code enhancement. Proper code documentation, version control (using platforms like GitHub), and adherence to naming conventions are essential to support long-term maintenance.

3.4.7 Portability Requirements

The application must be platform-independent and accessible through all modern browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. The system must also function efficiently on various devices including desktops, laptops, tablets, and mobile phones, maintaining full responsiveness and design consistency across all viewports.

3.4.8 Data Integrity Requirements

Data stored within the system, such as user profiles, Lawyer credentials, and appointment/payment histories, must remain consistent and error-free. The platform should implement validation at both the client and server sides to prevent duplication or missing entries. Backup mechanisms should be in place to restore data in case of failure, ensuring no permanent loss of critical legal information.

3.4.9 Availability Requirements

The system must be hosted on a reliable cloud-based infrastructure that guarantees 24/7 availability. Scheduled maintenance should be communicated in advance, and load balancers should distribute requests to prevent downtime during peak hours, particularly for the Live Chat (UC4) and Appointment Verification (UC9) services.

3.4.10 Extensibility Requirements

The architecture should allow for future enhancements without requiring major changes to the core system. The modular design should support easy integration of additional AI models (e.g., legal document analysis), new features (like video conferencing), and third-party APIs (e.g., for official registry verification). This ensures that the platform remains adaptable to future technological advancements in the legal tech space.

3.4.11 Backup and Recovery Requirements

The Law Sphere system must automatically perform periodic backups of critical data, including Lawyer records, user accounts, and financial histories. Backup data should be securely stored and easily retrievable in the event of a system crash or server malfunction. The recovery process should restore full system functionality with minimal data loss.

3.4.12 Compliance and Legal Requirements

The platform must comply with international data protection and privacy standards such as GDPR and region-specific legal practice regulations. User data and case details should only be collected for legitimate purposes and must not be shared without consent. All platform features, including Live Chat (UC4), must comply with professional confidentiality guidelines.

3.4.14 Load Handling Requirements

The system should handle concurrent access from a minimum of 500 users simultaneously without noticeable delay or service degradation. Performance testing tools should be used to verify load performance during the deployment phase, with a focus on high-traffic areas like search functionality and appointment scheduling.

Chapter 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

In the Design and Architecture phase of the Law Sphere Lawyer Management System, we establish the complete structural blueprint of the application and outline how its components interact to deliver a seamless service experience. This phase focuses on designing intuitive user interfaces for Clients, Lawyers, and Administrators while also defining a robust backend architecture that efficiently manages lawyer profiles, appointment scheduling, user authentication, and critical financial workflows.

4.1 System Architecture

The Law Sphere Platform follows a three-tier architecture that ensures modularity, scalability, and efficient data handling. Each layer is responsible for distinct operations, allowing independent updates and maintenance without disrupting the entire system.

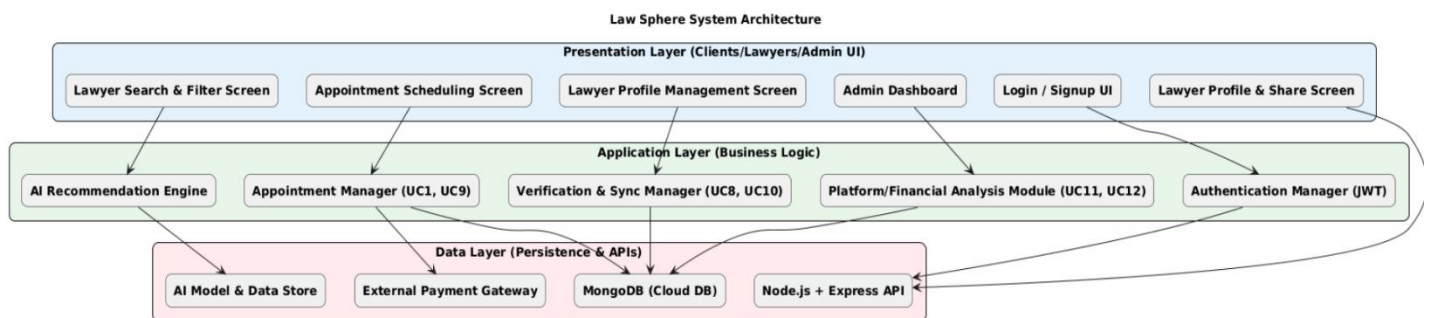


Figure 4.1: Law Sphere System architecture

Explanation

	Responsibility	Actors Involved
Presentation Layer	Manages all user-facing interfaces, including user registration and lawyer browsing.	Clients, Lawyers, Admin
Application Layer	Responsible for the system's core business logic, including AI recommendations, user authentication (UC7), appointment validation (UC9), payment verification (UC8), and data synchronization.	Clients, Lawyers, Admin
Data Layer	Manages the backend APIs, database connectivity, and AI data models, ensuring secure and efficient data flow for profiles, case history, and financial transactions.	System

4.2 Data Representation

The data model for the Law Sphere Platform is designed using a document-oriented schema (e.g., MongoDB) to handle the flexible structure of legal specializations and user-specific records. Each collection represents a major system entity such as Users, Lawyers, Appointments, and Financials.

This structure ensures flexible data handling for AI recommendations, performance analysis (UC11), and secure user operation

4.2.1 Key Entities and Relationships

- **User (Client)** – Stores basic authentication and profile information for individuals seeking legal services.
- **Lawyer** – Stores professional credentials, specializations, fee structure, availability, and Admin approval status (UC10).
- **Appointment** – Manages scheduled consultations, including date, time, status (Pending, Confirmed), and links to the Client and Lawyer.
- **Financials** – Manages all payment records, client fees, lawyer earnings, and payout verification status (UC8, UC12).
- **AI Model** – Stores processed data for recommending Lawyers and analyzing platform trends (UC11).

Relationships:

- One **Client** can schedule multiple **Appointments**.
- One **Lawyer** can have multiple **Appointments**.
- Each **Lawyer** is linked to multiple **financial** records (earnings).
- The **Admin Dashboard** retrieves aggregated data from all collections for reporting (UC11).

4.2.2 ER Diagram

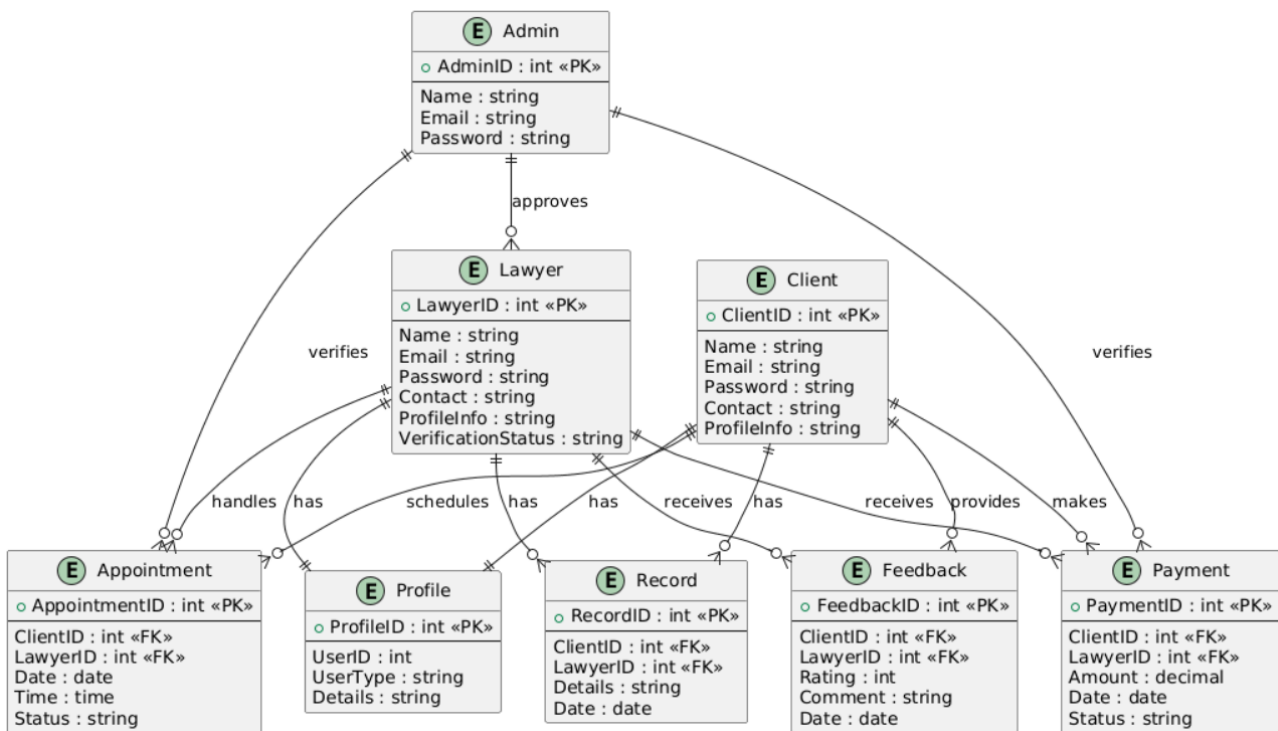


Figure 4.3: Law Sphere ER Diagram

Explanation

The Client entity represents users seeking legal assistance and booking appointments. The Lawyer entity contains profiles of lawyers offering legal services. The Appointment entity connects clients and lawyers for scheduling meetings. The Payment entity manages financial transactions between clients and lawyers. The Feedback entity allows clients to provide ratings and comments on lawyers' services. The Admin entity oversees verification, approvals, dispute handling, and system monitoring. The Profile entity stores additional information for both clients and lawyers.

4.3 Process Flow/Representation

The **Law Sphere** enables clients to explore, book, and interact with lawyers efficiently. Clients can register/login, schedule appointments, make payments, provide feedback, and chat live with lawyers.

4.3.1 Activity Diagrams

These diagrams effectively illustrate the sequence of steps involved in each workflow, highlight key decision points. By visualizing the system's workflow, activity diagrams enhance communication between developers and stakeholders, support more accurate planning and feature implementation, and ensure that the platform's overall logic and interactions are clearly understood throughout the development process. They also reduce ambiguity during development, helping teams detect potential issues early. Additionally, these diagrams provide a strong foundation for building a system that is efficient, user-friendly, and aligned with real user needs.

Activity diagrams illustrate workflows for key operations, decision points, and parallel processes in the system. They ensure clear communication between developers and stakeholders, support planning, and highlight potential issues early.

4.3.1.1 Client Activities:

Clients can register/login, schedule appointments, make payments, provide feedback, view records, and chat with lawyers.

The interface is intuitive, responsive, and guides users with prompts and feedback, improving user engagement and satisfaction.

4.3.1.2 Administrator Activities:

Admins manage lawyer profiles, verify appointments and payments, handle financial disputes, generate reports, and monitor system logs.

Role-based permissions ensure security, and real-time updates allow admins to respond quickly to system changes.

4.3.1.3 User Login and Registration Activity Diagram

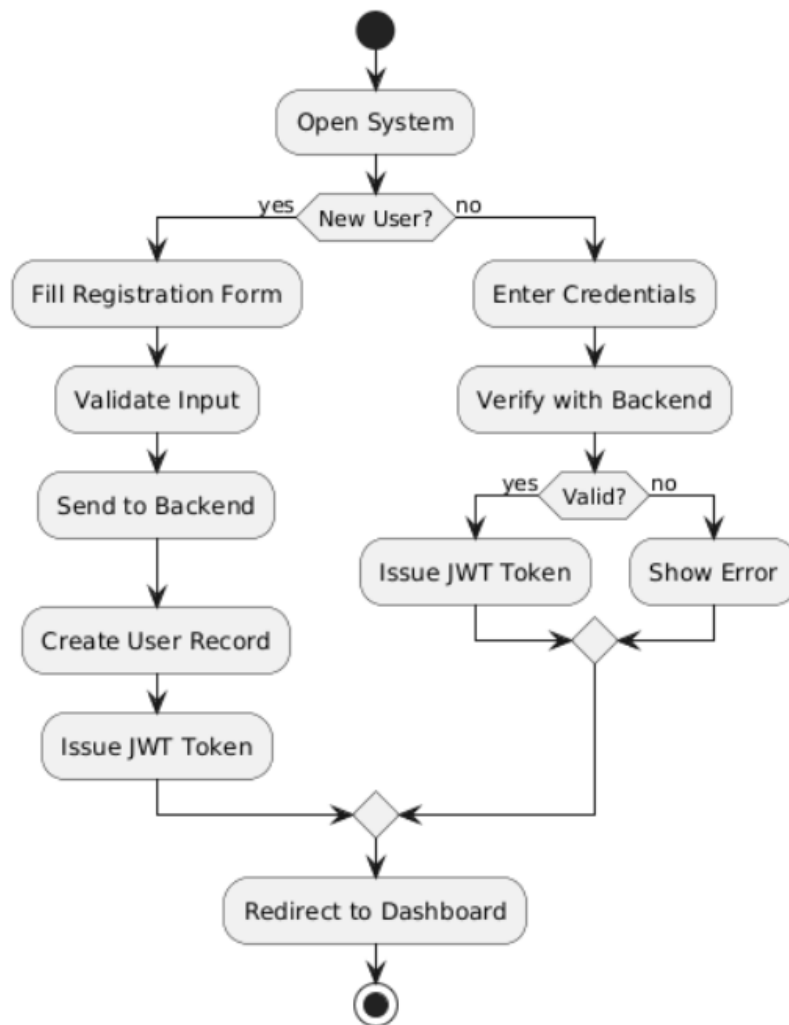


Figure 4.4: User Login and Registration Activity Diagram

Figure 4.3 illustrates the user authentication workflow in the Lawyer Management System. The process begins when a client or lawyer opens the system and selects either the Register or Login option. For registration, the system collects required profile details (name, email, password, contact) and performs validation checks. Once validated, the registration request is sent to the backend, which creates the user record in the database and issues a secure JWT token. During login, credentials are forwarded from the frontend to the backend for

verification. The authentication manager validates the credentials against stored hashes and issues a JWT upon success. Error cases — such as duplicate email, weak password, or invalid credentials — are handled by providing appropriate messages and retry options. After successful authentication, the user is redirected to the main dashboard.

4.3.1.4 Schedule Appointment Activity Diagram

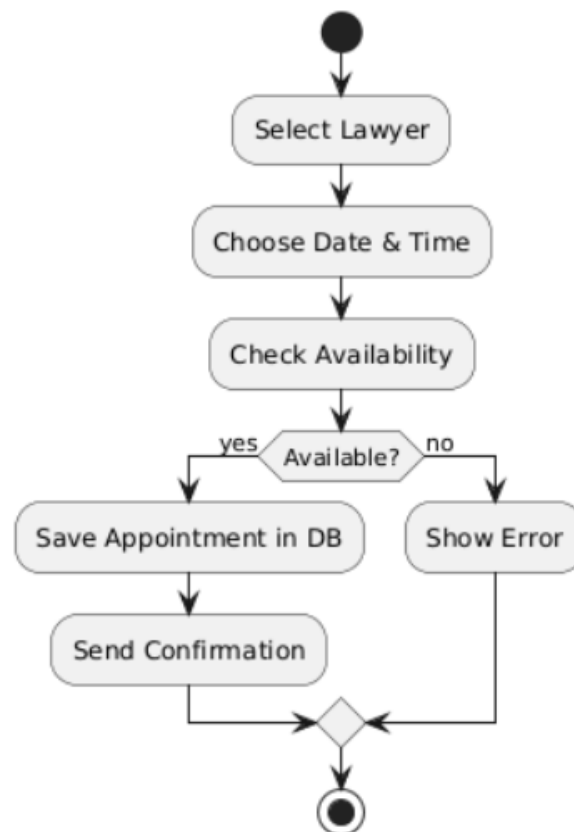


Figure 4.4: *Schedule Appointment Activity Diagram*

Figure 4.4 depicts the workflow for scheduling an appointment between a client and a lawyer. The process begins when a client selects a lawyer and chooses a preferred date and time. The system checks the lawyer’s availability to ensure no conflicts. If the selected slot is available, the appointment is recorded in the database, and a confirmation notification is sent to both the

client and the lawyer. In case of conflicts or invalid inputs, the system provides appropriate error messages and allows the client to select a different time. This workflow ensures a seamless and reliable appointment scheduling process, maintaining accurate calendar management for lawyers and convenience for clients.

4.3.1.5 Make Payment Activity Diagram

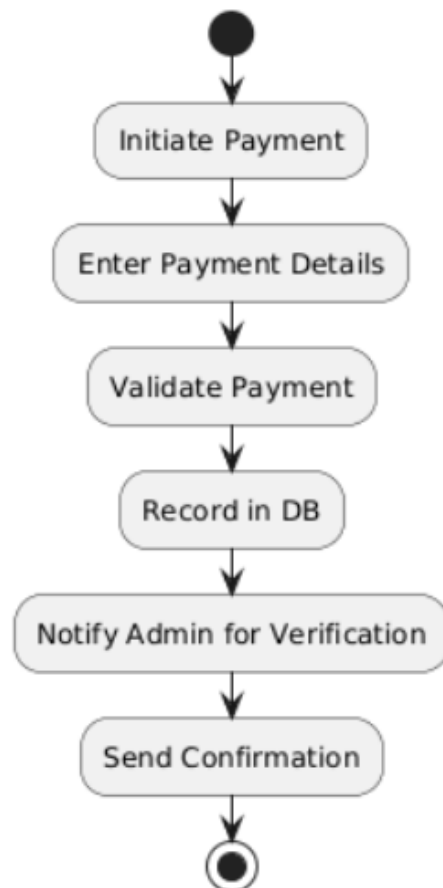


Figure 4.5: Make Payment Activity Diagram

Figure 4.5 illustrates the payment workflow in the Lawyer Management System. Once a client initiates a payment for a service, they provide payment details, which the system validates for correctness and completeness. The payment is then recorded in the database, and the admin is notified for verification. Upon successful verification, a confirmation is sent to the client and the corresponding lawyer. Error handling is incorporated to manage cases such as invalid payment details, insufficient funds, or verification failure. This ensures secure and reliable financial transactions within the system.

4.3.1.6 Provide Feedback Activity Diagram

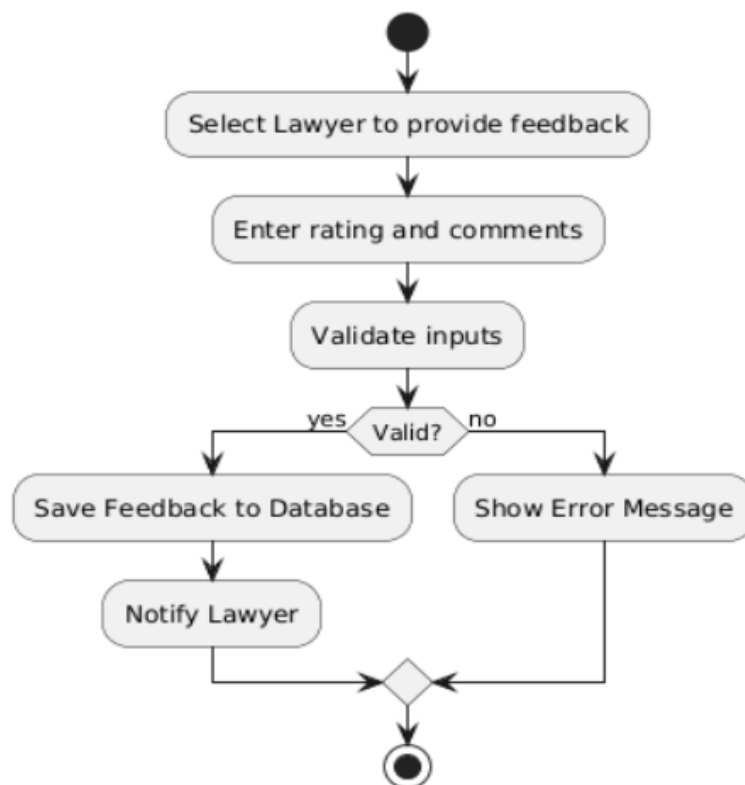


Figure 4.6: Provide Feedback Activity Diagram

Figure 4.6 illustrates the workflow for clients providing feedback on a lawyer's service. The client selects the lawyer, enters a rating and comments, and the system validates the inputs. If

valid, the feedback is stored in the database and the lawyer is notified. In case of invalid inputs, such as missing rating or exceeding character limits, an error message is displayed and the client is prompted to correct the entry. This workflow ensures that feedback is accurately recorded and communicated, enhancing service quality and accountability.

4.3.1.7 Live Chat Activity Diagram

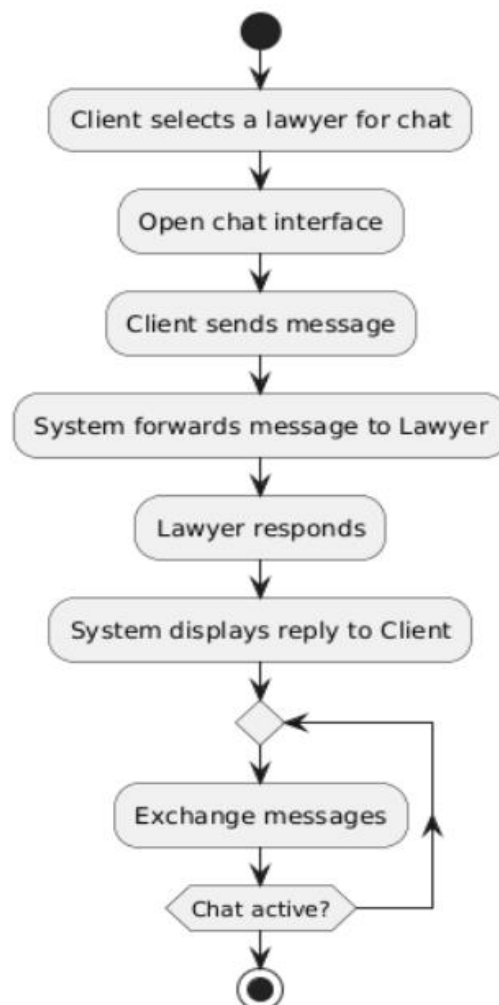


Figure 4.7: Live Chat Activity Diagram

Figure 4.7 depicts the live chat workflow between a client and a lawyer. The client selects a lawyer and opens the chat interface. Messages sent by the client are forwarded by the

system to the lawyer, who can respond, and the reply is displayed to the client in real-time. This loop continues until the chat session ends. The diagram demonstrates a continuous, interactive communication process that enables clients and lawyers to clarify queries, improving engagement and service responsiveness.

4.3.1.8 Admin Manage Dashboard Activity Diagram

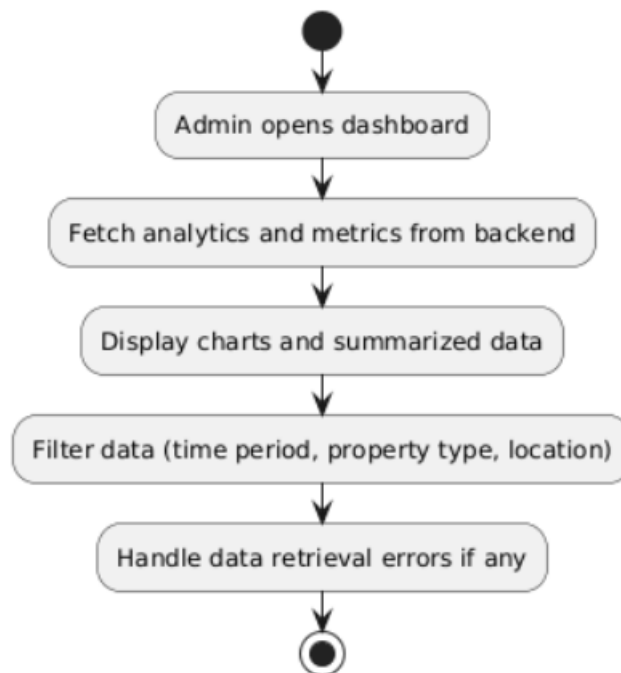


Figure 4.8: Manage Dashboard Activity Diagram

Figure 4.8 shows the workflow for admin dashboard management. The admin logs into the system, and the dashboard fetches real-time data such as appointments, payments, and feedback from the backend. The admin can filter data by type, time, or role, and export reports when necessary. Error handling is included for cases where data retrieval fails. This workflow ensures that administrators have a centralized view of system operations, enabling efficient monitoring and data-driven decision-making.

4.4 Design Models

4.4.1 Sequence Diagram

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus, and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent.

4.4.1.1 Sequence Diagram for User Registration and Login

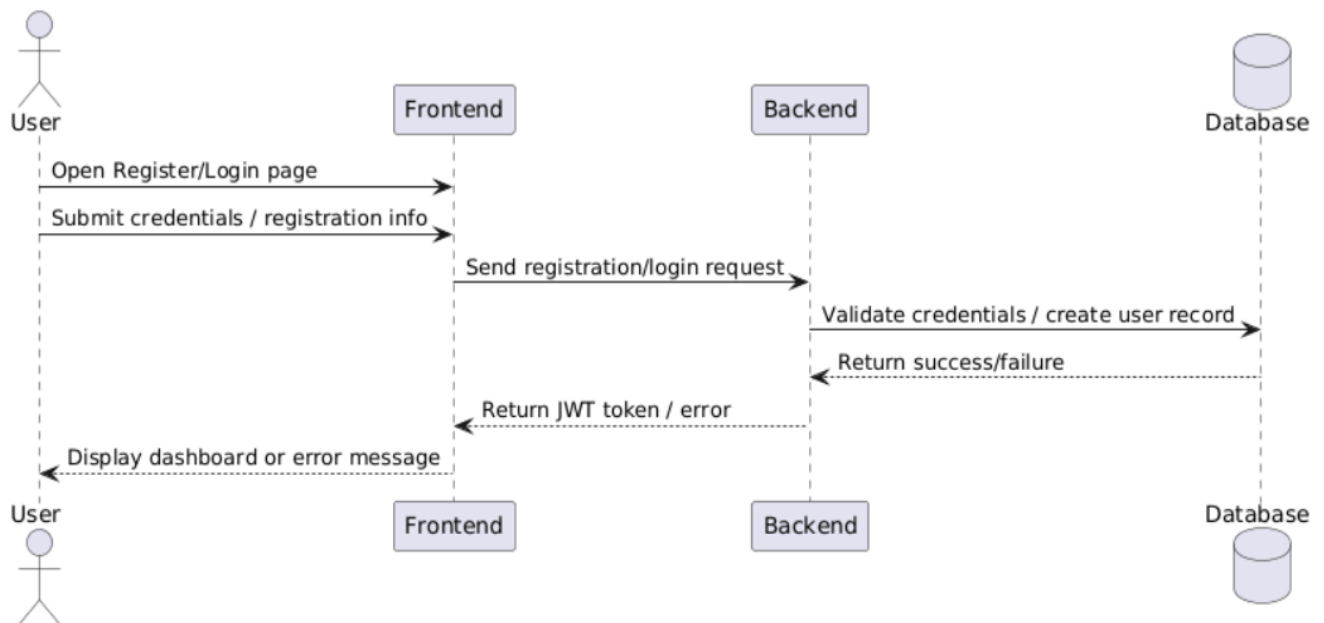


Figure 4.9: *Sequence Diagram for User Registration and Login*

Figure 4.9 illustrates the user registration and login sequence. Users submit their credentials or profile information through the frontend, which forwards the request to the backend API. The backend validates credentials or creates a new user record in the database. Upon success, a JWT token is returned to the frontend, granting secure access to the main dashboard. Errors such as duplicate emails or invalid credentials are handled and displayed to the user. This sequence ensures secure and validated access to the platform.

4.4.1.2 Sequence Diagram for Provide Feedback

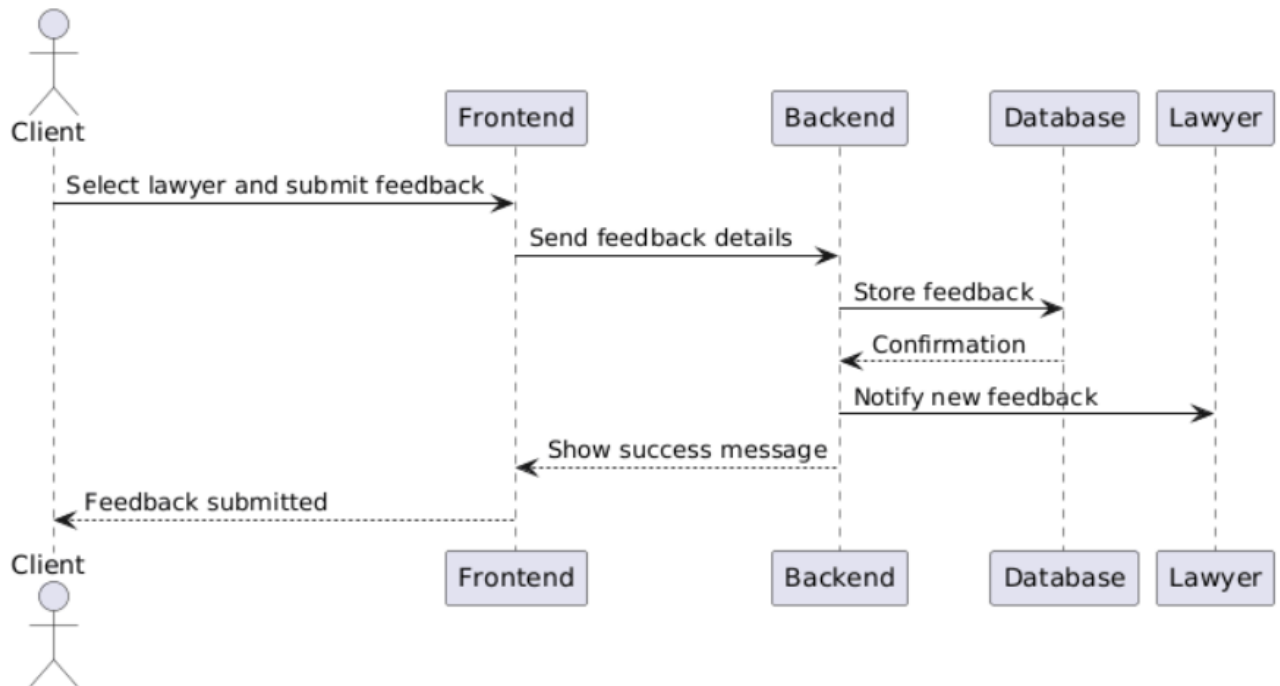


Figure 4.10: *Sequence Diagram for Provide Feedback*

Figure 4.10 illustrates the feedback submission sequence. The client selects a lawyer and submits feedback through the frontend. The backend stores the feedback in the database and sends a notification to the lawyer. The frontend then displays a success message to the client. This ensures proper recording of feedback and timely communication to lawyers for improved service quality.

4.4.1.3 Sequence Diagram for Book Appointment

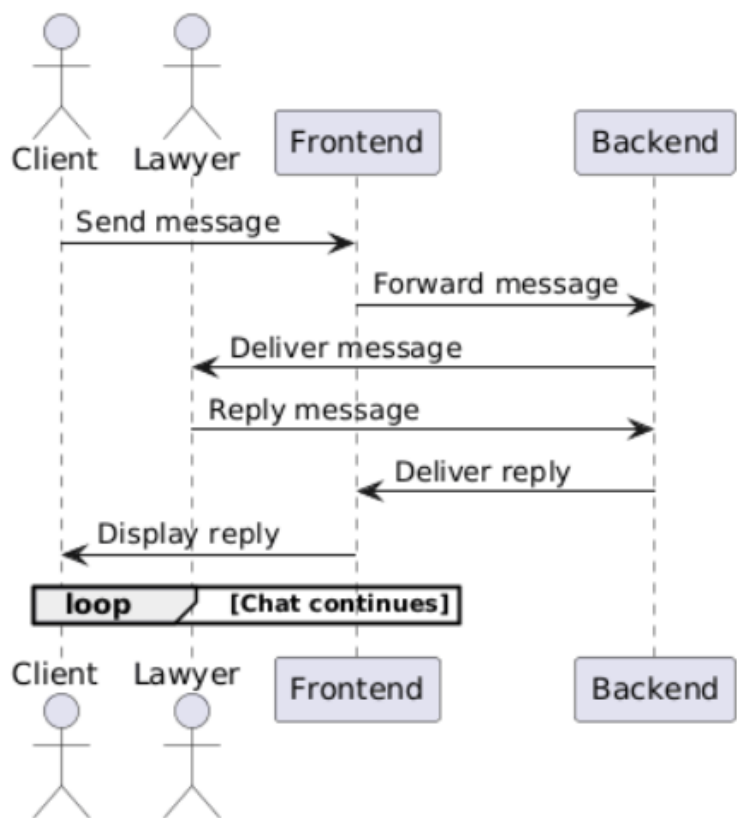


Figure 4.11: Book Appointment Sequence Diagram

Figure 4.11 shows the live chat interaction between a client and a lawyer. Messages from the client are sent to the backend and delivered to the lawyer, whose responses are routed back through the backend to the client. This sequence continues until the chat session ends. The workflow highlights real-time communication and ensures messages are delivered accurately and promptly.

4.4.1.4 Sequence Diagram for Admin Approve Lawyer Profile

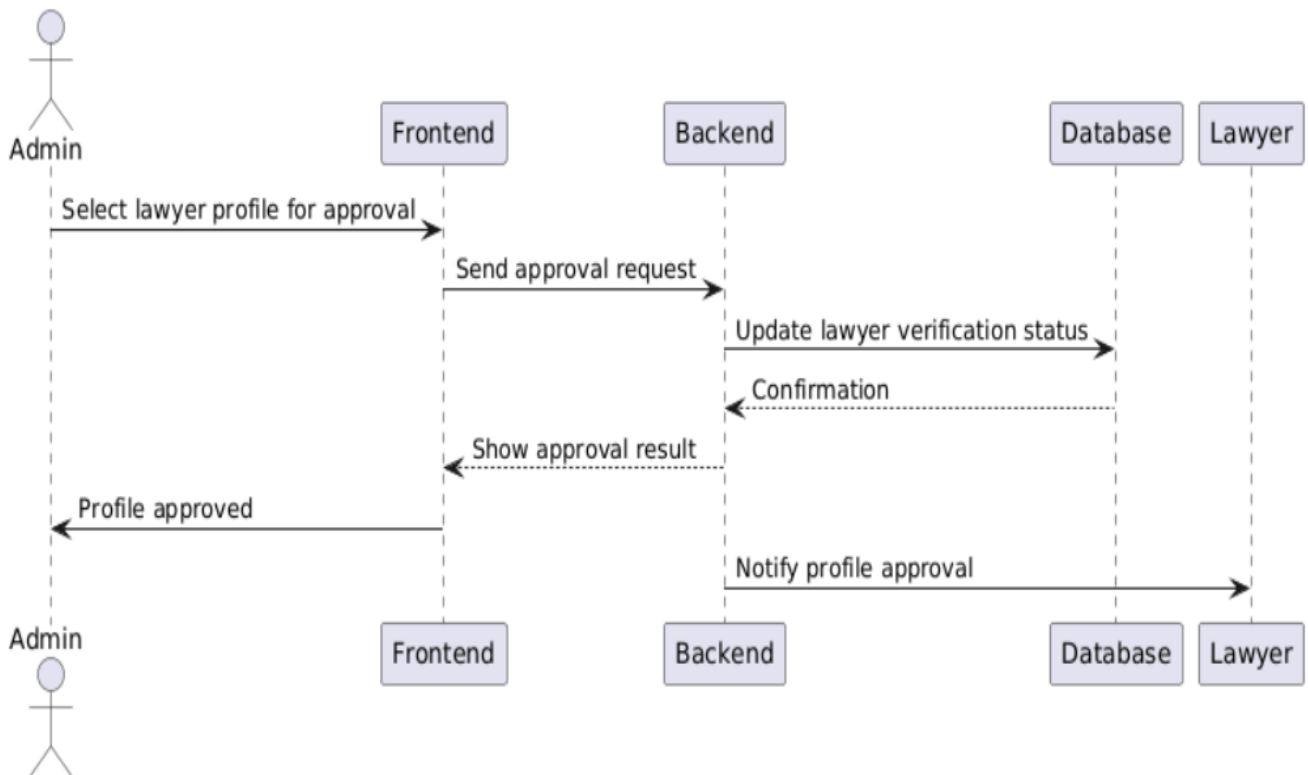


Figure 4.12: Sequence Diagram for Admin Approve Lawyer Profile

Figure 4.12 illustrates the workflow for admin approval of a lawyer's profile. The admin selects a profile, sends an approval request through the frontend, and the backend updates the verification status in the database. A notification is sent to the lawyer, and the frontend confirms the approval to the admin. This sequence ensures that only verified lawyers are active on the platform, maintaining service integrity.

4.4.1.5 Sequence Diagram for Admin Manage Dashboard

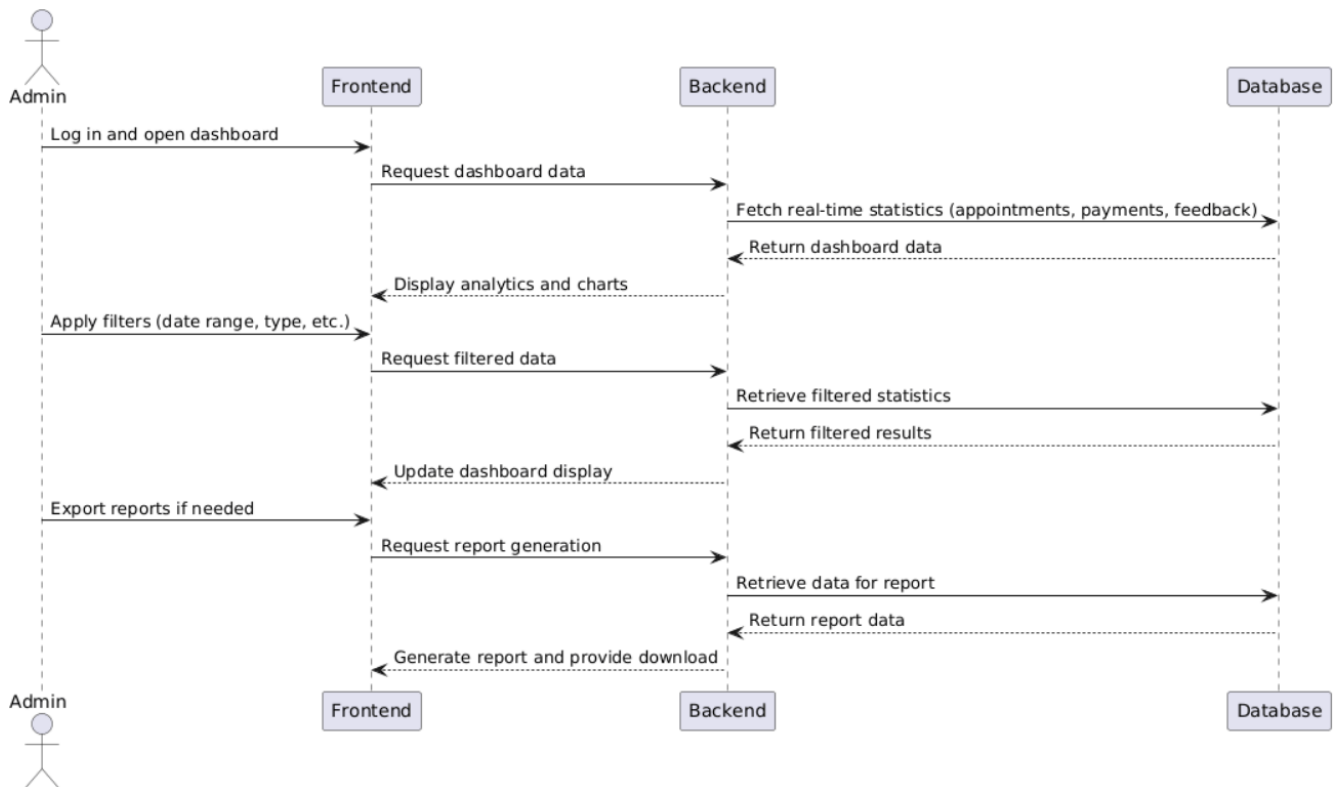


Figure 4.13: Admin Manage Dashboard Sequence Diagram

Figure 4.13 illustrates the workflow for managing the admin dashboard in the Lawyer Management System. The process begins when an admin logs into the system and opens the dashboard interface. The frontend requests real-time data from the backend, which retrieves statistics from the database, including appointments, payments, feedback, and system logs. The backend sends this data back to the frontend, where analytics and charts are displayed for the admin.

4.4.2 Class Diagram

A class diagram is a static structure diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships. It is widely used in object-oriented systems to visualize, document, and construct executable code [6]. Class diagrams describe the static view of an application and are crucial for modeling system functionalities and collaborations.

4.4.2.1 Class Diagram

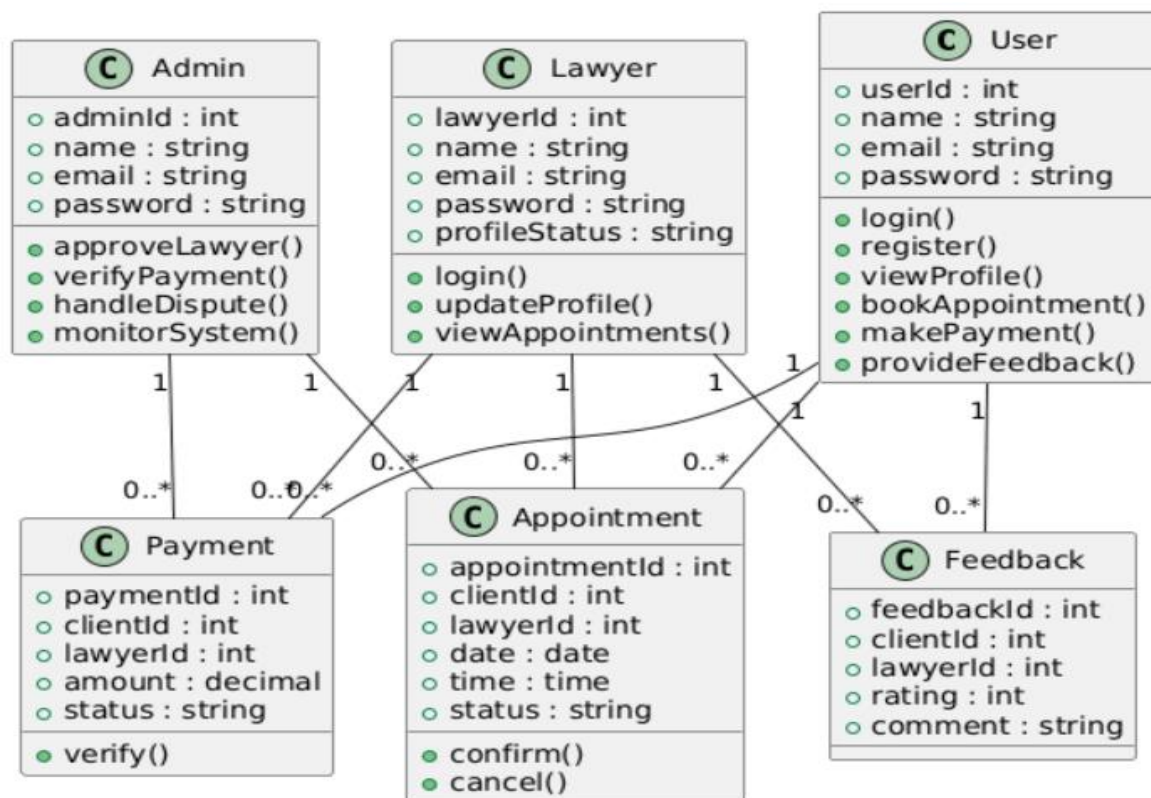


Figure 4.14: Class Diagram

Figure 4.14 shows the class diagram representing the core static structure of the Lawyer Management System. The **User** class defines attributes and methods for client operations such as registering, logging in, booking appointments, making payments, and providing feedback. The class diagram illustrates the core **entities** and their **relationships** in the Lawsphere

platform: Methods allow clients to register, login, view and update their profile, schedule appointments, make payments, provide feedback, and chat live with lawyers. **Admin Class** represents administrators managing the overall system operations. Attributes include adminId, name, email, and password. Methods allow admins to login, approve or reject lawyer profiles, verify payments, handle disputes, monitor system logs, and manage the dashboard for analytics and reporting. **Appointment Class** represents the appointments booked by clients with lawyers. Attributes include appointmentId, clientId, lawyerId, date, time, and status. Methods include confirmAppointment(), cancelAppointment(), and rescheduleAppointment(). Clients can book multiple appointment. **Payment Class** represents payments made by clients to lawyers for services. Attributes include paymentId, clientId, lawyerId, amount, date, and status. Methods include initiatePayment() and verifyPayment(). A client can make multiple payments; each payment is associated with a single lawyer. Admins verify payments. **Feedback Class** represents ratings and comments provided by clients for lawyers. Attributes include feedbackId, clientId, lawyerId, rating, comment, and date. Methods include submitFeedback() and viewFeedback(). **Record Class** represents legal records or history of interactions between clients and lawyers. Attributes include recordId, clientId, lawyerId, details, and date. Methods include addRecord() and viewRecord(). **Profile Class** represents additional profile details for both clients and lawyers. Attributes include profileId, userId, userType, and details. Methods include updateProfile() and viewProfile().

Relationships:

- Client → Appointment: A client can book multiple appointments.
- Appointment → Lawyer: Each appointment is associated with a single lawyer.
- Client → Payment: A client can make multiple payments.
- Lawyer → Payment: Each payment is linked to a single lawyer.
- Client → Feedback: A client can provide multiple feedback entries.
- Lawyer → Feedback: Each feedback entry is associated with a single lawyer.

Chapter 5

IMPLEMENTATION

5 Implementation

In the implementation phase of the Law Sphere, we have developed algorithms to power the core functionalities of the application, enabling secure user authentication, real-time appointment scheduling, feedback handling, live chat, and administrative operations. These algorithms ensure smooth interaction between clients, lawyers, and admins while maintaining data integrity and seamless user experience. AI-driven components assist admins in monitoring system activity and detecting patterns for reporting and decision-making.

5.1 Algorithm

5.1.1 User Authentication

Function: authenticateUser(email, password)

Input: email (string), password (string)

Output: user (object) or error (string)

Description:

This algorithm handles secure authentication for both Clients, Lawyers, and Admins in the Lawyer Management System. It uses bcrypt for password hashing and JWT tokens for session management, ensuring that only verified users can access protected features.

5.1.2 Schedule Appointment

Function: bookAppointment(userId, lawyerId, date, time)

Input: userId (string), lawyerId (string), date (string), time (string)

Output: appointmentConfirmation (object) or error (string)

Description:

This algorithm allows clients to schedule appointments with lawyers, checking availability and maintaining accurate scheduling records.

Pseudocode:

1. Check if the selected lawyer is available at the requested date and time.

2. If available, create a new appointment record in the database.
3. Update the lawyer's schedule to mark the slot as booked.
4. Notify both the client and lawyer of the confirmed appointment.
5. If not available, return "Selected slot not available."

5.1.3 Payment Processing

Function: processPayment(userId, lawyerId, amount, paymentMethod)

Input: userId (string), lawyerId (string), amount (number), paymentMethod (string)

Output: paymentConfirmation (object) or error (string)

Description:

This algorithm manages payments from clients to lawyers for booked services, ensuring secure and verified transactions.

Pseudocode:

1. Collect payment details from the client.
2. Validate payment information and check for sufficient funds.
3. Record the payment in the database with status "Pending."
4. Admin verifies the payment; if successful, update status to "Completed."
5. Notify the client and lawyer of successful payment.
6. If verification fails, return an error message

5.1.4 Feedback Submission

Function: submitFeedback(clientId, lawyerId, rating, comment)

Input: clientId (string), lawyerId (string), rating (number), comment (string)

Output: success (Boolean) or error (string)

Description:

This algorithm allows clients to provide ratings and comments for lawyers, enhancing service quality and accountability.

Pseudocode:

1. Validate feedback inputs (rating within range, comment length).
2. Store feedback in the database linked to the client and lawyer.
3. Notify the lawyer of new feedback.
4. Return success confirmation or an error message if validation fails.

5.1.5 Live Chat Management

Function: sendMessage(senderId, receiverId, message)

Input: senderId (string), receiverId (string), message (string)

Output: messageStatus (object) or error (string)

Description:

This algorithm facilitates real-time chat between clients and lawyers.

Pseudocode:

1. Validate sender and receiver IDs.
2. Store the message in the database with timestamp.
3. Forward the message to the intended receiver.
4. Confirm message delivery to the sender.
5. Handle errors such as invalid IDs or server issues.

5.1.6 Admin Verification & Approval

Function: verifyLawyerProfile(adminId, lawyerId, action)

Input: adminId (string), lawyerId (string), action (string — approve/reject)

Output: success (Boolean) or error (string)

Description:

This algorithm allows admins to approve or reject lawyer profiles to maintain platform integrity.

Pseudocode:

1. Check admin permissions.
2. If action is "approve," update lawyer verification status to approved.
3. If action is "reject," mark profile as rejected and notify lawyer.
4. Return success confirmation or error message.

5.1.8 Dashboard Analytics

Function: generateDashboardData()

Input: None

Output: dashboardMetrics (object)

Description:

This algorithm collects platform statistics such as total clients, lawyers, appointments, payments, and disputes for the admin dashboard.

Pseudocode:

1. Query the database for total clients, lawyers, and active appointments.
2. Aggregate payments and dispute records.
3. Calculate performance metrics such as response times and feedback scores.
4. Return summarized data for dashboard visualization.

5.1.10 Admin Management

Function: manageAdmin(mainAdminId, action, adminId)

Input: mainAdminId (string), action (string — add/edit/remove), adminId (string)

Output: success (Boolean) or error (string)

Description:

This algorithm allows the main administrator to manage other admins in the system.

Pseudocode:

1. Verify that the current user has main admin rights.
2. If action is "add," create a new admin record.
3. If action is "edit," update the selected admin details.
4. If action is "remove," delete the admin record.
5. Return success confirmation or appropriate error message.

These are simplified pseudocode representations of the algorithms for each core component of the Law Sphere. The actual implementation may include additional error handling, input validation, and integration with frontend components. It also involves real-time interactions between the user interface, backend APIs, and the cloud-based MongoDB Atlas database, along with AI-based processing.

5.2 External APIs

Table 15.2.1: External APIs

Library / API	Purpose
Axios	Handles REST API requests between frontend and backend.
Express.js	Framework for building RESTful backend APIs and routing.
MongoDB Atlas	Cloud database for storing clients, lawyers, appointments, payments, and feedback.

Mongoose	ODM library to model and interact with MongoDB data.
Cloudinary / Imagekit API	For uploading and managing lawyer profile images and documents.
JWT (JSON Web Token)	Used for secure user authentication and session handling.
Bcrypt.js	Password hashing and verification for secure login.
OpenAI / ML Model API	Provides AI-powered analytics for appointment trends, lawyer performance insights, and dispute resolution support.
Nodemailer	Sends appointment confirmations, payment receipts, and notification emails.

Table 15.2.1: lists the key libraries and APIs utilized in the Lawyer Management System along with their roles. Axios handles communication between the frontend and backend, while Express.js provides a structured framework for RESTful APIs. MongoDB Atlas serves as the cloud-based database, and Mongoose simplifies database interactions. Cloudinary or Imagekit handles lawyer profile images and document uploads. JWT and Bcrypt.js ensure secure authentication and password management. OpenAI or other ML models provide AI-powered analytics for appointment scheduling, dispute handling, and performance insights. Nodemailer sends notifications and confirmations.

5.3 User Interface

The Lawyer Management System is designed with a **user-centric approach**, ensuring that clients, lawyers, and admins can easily navigate and utilize its features without technical expertise. The interface is **modern, responsive, and intuitive**, allowing all user types to manage their tasks efficiently. Each section of the platform is organized into **clear workflows** that mirror real-world legal consultation, appointment management, and administrative processes.

5.3.1 Lawyer Account Creation Screen

Lawyer Portal

Create Lawyer Account

Connect with clients and grow your legal practice

First Name

Last Name

Lawyer Type / Specialization

Experience (in years)

Bar Council Registration Number

Law Firm / Office Name

City / Location

Phone Number

CNIC / ID Number

License / Certificate Upload No file chosen

Profile Photo (optional) No file chosen

We host via ImageKit. Max 2MB, JPG/PNG recommended.

Username

Email Address

Password

Confirm Password

[Already have an account? Sign In](#)

[← Back to Registration Options](#)

Figure 5.1: Lawyer Account Creation Screen

Figure 5.1 The **Lawyer Portal Account Creation** screen captures all necessary professional and personal details required for a lawyer to join the platform. Mandatory fields include name, specialization, years of experience, Bar Council Registration Number, and CNIC/ID Number, ensuring profile authenticity (UC10). The form also includes an option to upload the license/certificate and sets up secure access with a unique username, email, and password. This process ensures only verified legal professionals are onboarded.

5.3.2 Home Screen

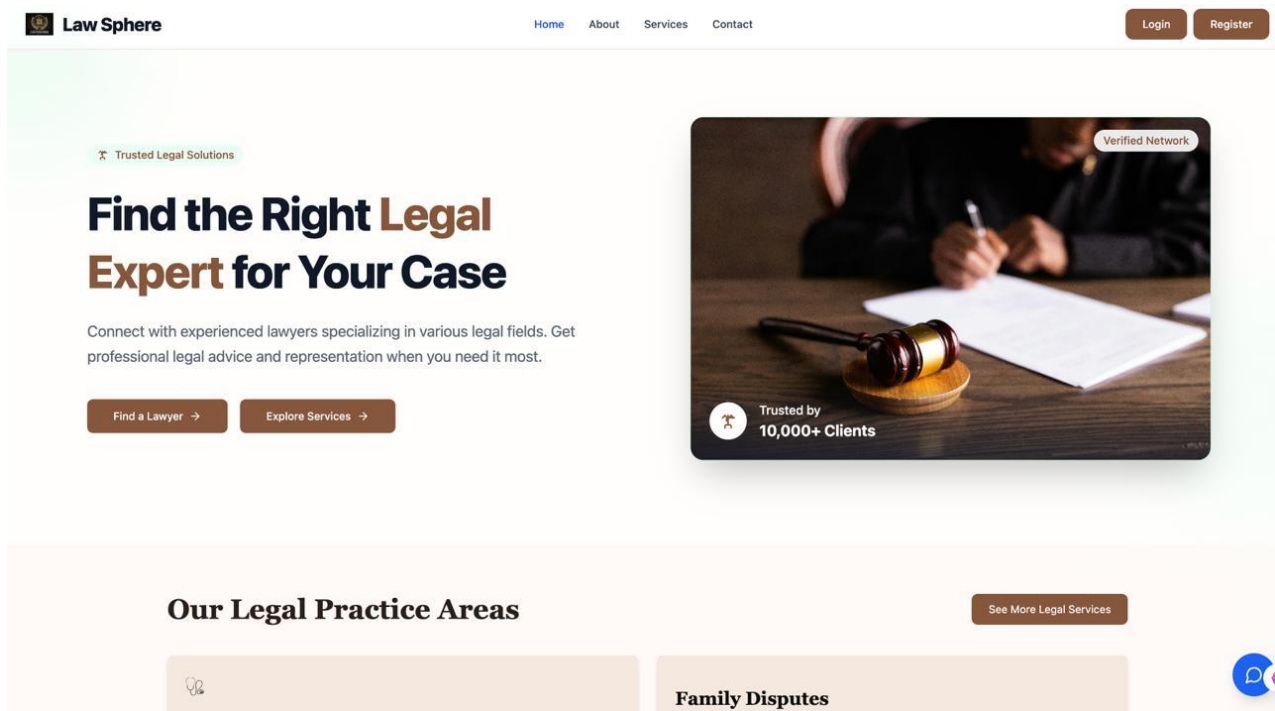


Figure 5.2: Home Screen

Figure 5.2: The **Law Sphere Homepage** serves as the primary gateway for clients, immediately establishing its value proposition: "Find the Right Legal Expert for Your Case." It features a clean, professional design that emphasizes a "Verified Network" and builds trust with a "Trusted by 10,000+ Clients" count. Key calls-to-action, such as "Find a Lawyer" and "Explore Services," guide the user directly into the search and specialization workflow.

5.3.3 About Us Screen

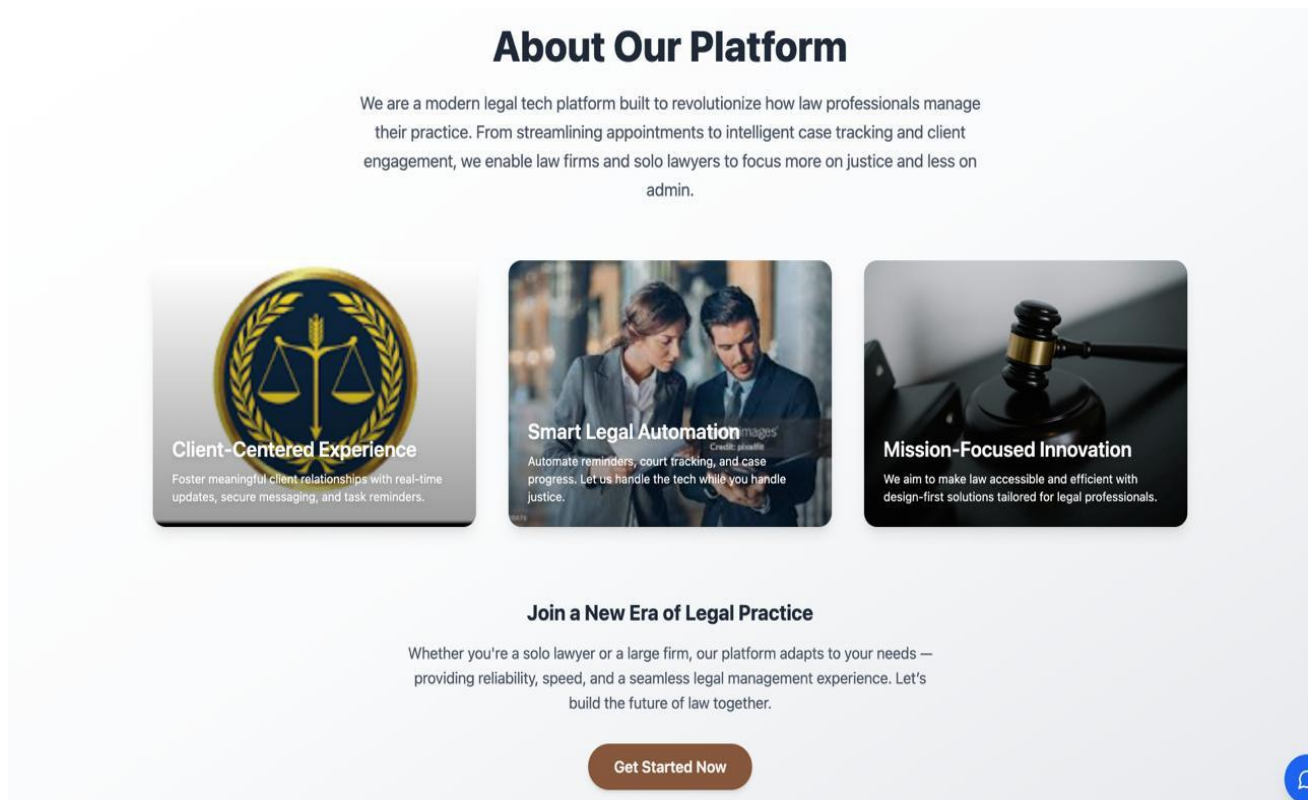


Figure 5.3: About Us Screen

Figure 5.4: The "About Our Platform" section details the core commitment of Law Sphere to streamline legal practice. It highlights three key features: Client-Centered Experience (fostering relationships with real-time updates), Smart Legal Automation (automating reminders and case tracking), and Mission-Focused Innovation (making law accessible). This section justifies the system's role in reducing administrative overhead for lawyers and improving service for clients.

5.3.4 Contact Us Screen

Contact Information
Multiple ways to reach our platform support team

Email Support
support@legalconnect.com
Get a response within 24 hours

Phone Support
+1 (555) 123-4567
Mon-Fri, 9 AM - 6 PM EST

Head Office
123 Legal Street, Suite 400
New York, NY 10001

Business Hours
Monday - Friday
9:00 AM - 6:00 PM EST

Support Categories
Quick links to common platform support topics

- Account & Profile**
Profile setup, verification, settings
- Technical Support**
Bug reports, login issues, errors
- Billing & Payments**
Subscription, invoices, refunds
- Matching & Connections**
Lawyer-client matching process
- Privacy & Security**
Data protection, account security
- Feature Requests**
Suggestions and improvements

Connect With Us
Follow us on social media for updates and legal industry insights

LinkedIn Twitter Facebook Instagram

Platform Support Form
Need help with the platform? Fill out this form and our support team will assist you.
Note: This form is for platform support only, not for legal case inquiries.

First Name *
John

Last Name *
Doe

Email Address *
john.doe@example.com

I am a *
Select your role

Support Category *
Select a category

Subject *
Brief description of your issue

Message *
Please provide detailed information about your platform support request...

Send Message

By submitting this form, you agree to our Terms of Service and Privacy Policy

Figure 5.5: Contact Us Screen

Figure 5.5: The **Contact Information and Platform Support** page provides multiple avenues for users to reach the Law Sphere team. It offers clear contact details (email and phone support, physical address, and business hours) alongside a structured **Platform Support Form**. The form allows users to select their role and support category (e.g., Technical Support, Billing & Payments, Privacy & Security), ensuring that inquiries are routed efficiently. Additionally,

quick links to FAQs under **Support Categories** and social media links are provided for comprehensive user assistance.

5.3.5 Admin Dashboard Screen

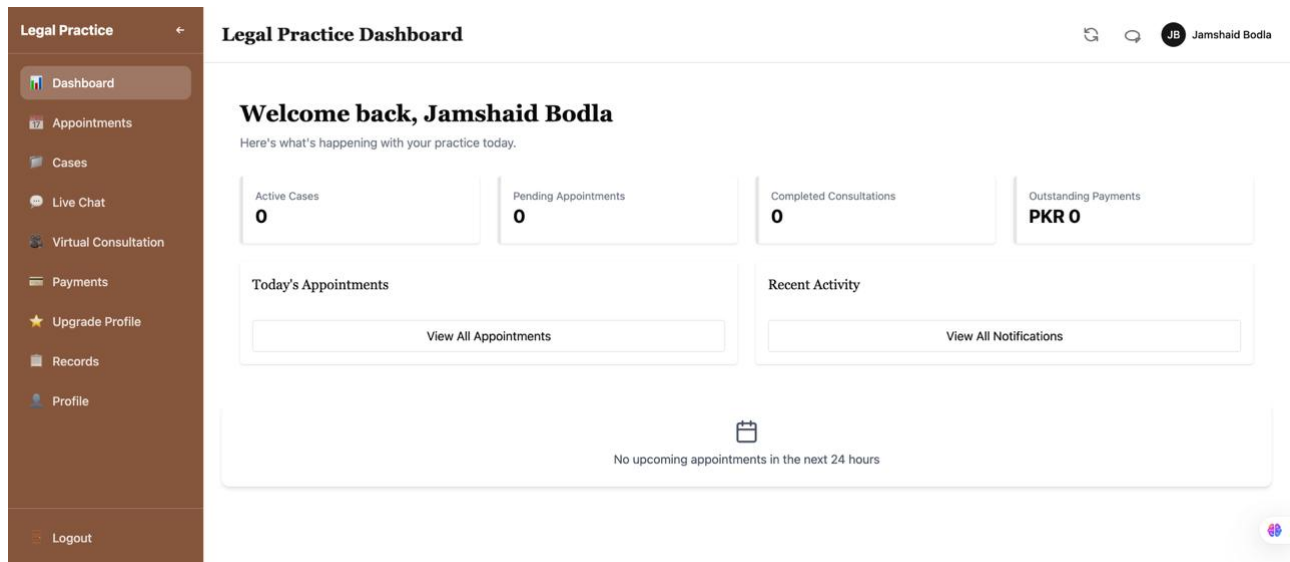


Figure 5.6: Admin Dashboard Screen

Figure 5.6: The Lawyer Practice Dashboard provides a centralized, real-time overview of the lawyer's professional activities. Key metrics summarized immediately upon login include Active Cases, Pending Appointments, Completed Consultations, and Payments. The sidebar navigation offers quick access to detailed sections for Appointments, Cases, Live Chat, Virtual Consultation, Payments, and Profile management, enabling the lawyer to manage their entire practice efficiently through one portal.

5.3.6 Case Management Screen

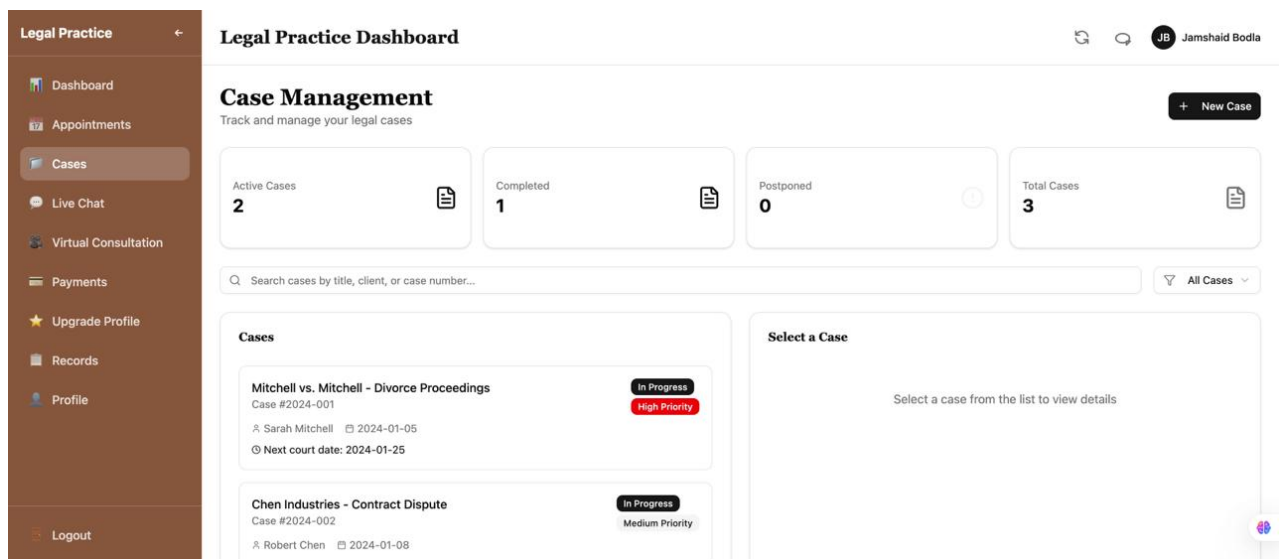


Figure 5.7: Case Management Screen

Figure 5.7: The Case Management screen provides the lawyer with tools to organize and track all active and completed legal matters. It displays key metrics such as Active Cases, Completed, Postponed, and Total Cases. The primary list shows case titles, unique case numbers, client names, and priority flags (e.g., High Priority), allowing for quick assessment. A dedicated right panel is reserved to Select a Case for viewing detailed history, documents, and upcoming court dates, which supports the overall administrative efficiency.

5.3.7 User Dashboard Screen

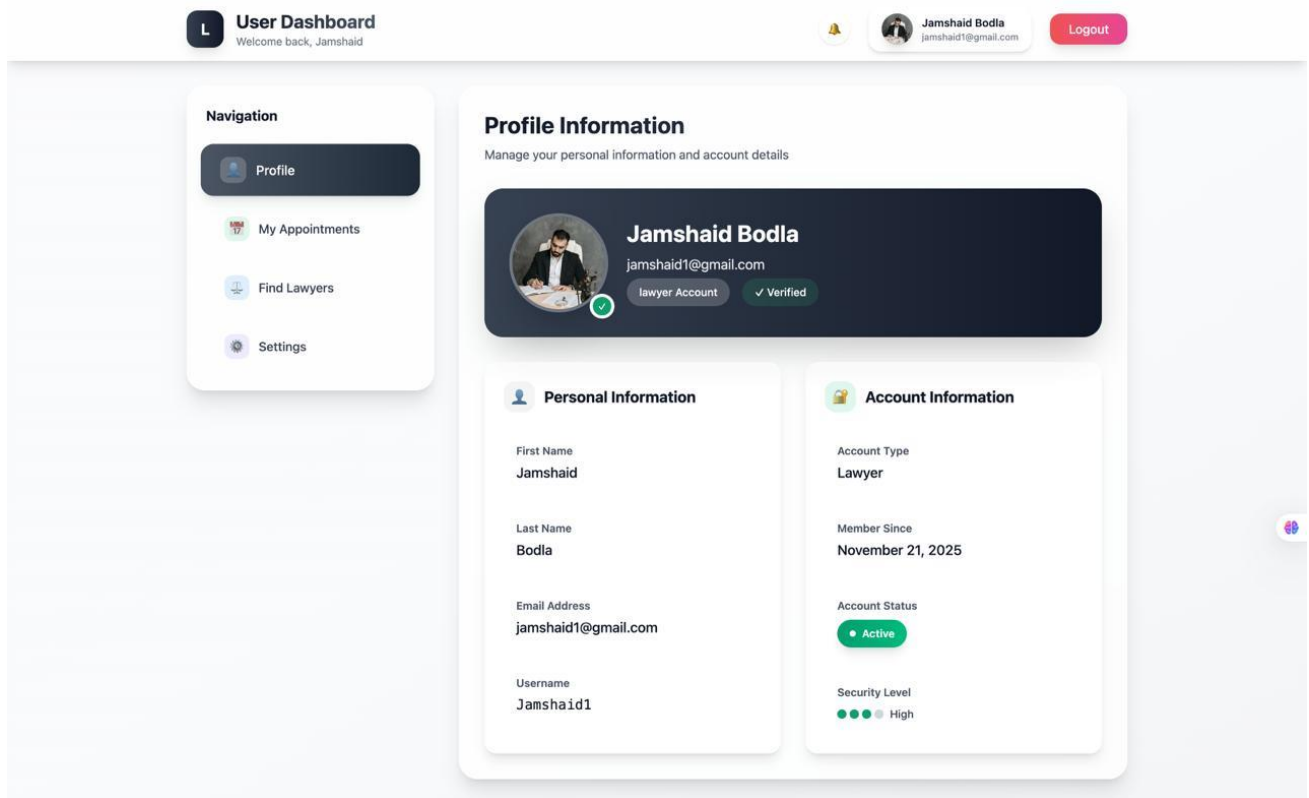


Figure 5.8: User Dashboard Screen

Figure 5.7: The User Dashboard - Profile Information screen provides the client with a secure area to manage their account details. The central focus is the Profile Information panel, which clearly displays the user's name, email, account type (shown here as a Lawyer Account, though this structure applies to Clients as well), and membership status. The clear navigation sidebar allows easy access to other core functions, including My Appointments (UC5), Find Lawyers (UC2), and Settings, making it the central hub for the client's platform interactions.

Chapter 6

TESTING AND EVALUATION

6 Testing and Evaluation

Testing plays a crucial role in ensuring that the **Law Sphere** functions correctly, meets user expectations, and delivers a seamless legal consultation and management experience. It is carried out at multiple stages of development to detect and fix errors early. Through systematic testing, the application's **performance, reliability, security, and usability** are continuously assessed. Following standard software engineering practices, testing is treated as a dedicated phase after implementation to allow for **objective evaluation**. By applying a structured testing approach, the system aims to enhance software quality, improve user experience, and eliminate major issues before deployment.

6.1 Manual Testing

Manual testing is one of the earliest and most effective methods used to identify issues in the system's interface, navigation flow, and core functionality. Testers interacted directly with the Lawyer Management System to verify that each feature behaves according to the system's requirements.

This testing was conducted without automated tools, which allowed direct observation of usability and functional performance. For instance, testers validated the login and registration processes for clients, lawyers, and admins, ensuring correct credential handling, secure access, and role-based permissions.

On the client side, manual testing verified whether users could search and filter lawyers by specialization, view detailed profiles, book appointments, make payments, provide feedback, and engage in live chat. On the lawyer side, testers ensured that profiles could be updated, appointments viewed and managed, and messages received in real-time.

For admins, manual testing focused on profile verification, approval of lawyer accounts, payment verification, dispute handling, and dashboard monitoring. Edge cases such as invalid

inputs, duplicate entries, appointment conflicts, and network interruptions were tested to assess robustness and error-handling capabilities.

Manual testing also examined **interface responsiveness across different browsers and devices**, ensuring the platform remained intuitive, consistent, and easy to navigate.

6.1.1 System Testing

System testing was performed after all modules were integrated to verify the overall behavior of the Lawyer Management System. The focus was on **performance, reliability, security, and interaction** between clients, lawyers, and admin components.

Table 6.1.1.1: System Testing Table

Test ID	Feature Tested	Description	Expected Result	Actual Result	Status
ST-01	User Authentication	Test login and registration for clients, lawyers, and admins	Successful login/signup	As expected	Pass
ST-02	Lawyer Search & Filtering	Verify search and filter functionalities	Displays relevant lawyer profiles	As expected	Pass
ST-03	AI Recommendations	Check AI module for appointment trends and analytics	Provides accurate suggestions and insights	As expected	Pass
ST-04	Appointment Booking	Verify booking workflow and	Appointment saved and	As expected	Pass

		schedule management	displayed to client & lawyer		
ST-05	Payment Processing	Test payment flow and verification	Payments recorded, confirmed, and notified	As expected	Pass

Table 6.1.1.1 summarizes the overall system testing results. Each test case corresponds to a major system function, such as login, AI recommendations, appointment booking, or payment verification.

The “Expected Result” defines how the system should ideally behave, while the “Actual Result” records the output observed during testing. The “Status” column indicates whether the system passed or failed each test.

The consistent “Pass” results demonstrate that all core components of the Lawyer Management System are stable, reliable, and capable of handling real-world usage scenarios effectively.

6.1.2 Unit Testing

Unit testing involves verifying the functionality of individual components or modules of the system in isolation. Each function or unit was tested independently to ensure that it performs its specific task correctly before being integrated into the main system.

For the **Lawyer Management System**, unit testing focused on critical modules such as:

Authentication Module: Tested the validation of client, lawyer, and admin login credentials, ensuring role-based access and secure authentication.

Lawyer Management Module: Verified that adding, editing, and approving lawyer profiles worked correctly and that profile data was stored accurately in the database.

Appointment System: Tested the creation, updating, and cancellation of appointments between clients and lawyers, including conflict detection for overlapping time slots.

Payment Module: Ensured that payment processing, verification, and confirmation workflows functioned accurately.

Feedback Module: Checked that clients could submit feedback and that it was correctly stored and retrievable by lawyers and admins.

Dashboard Module: Validated data visualization and admin access to system statistics such as total users, appointments, payments, and dispute reports.

These tests were carried out using **manual verification** as well as **lightweight testing scripts**. Unit testing helped identify early-stage bugs before integration, improving overall software **reliability, maintainability, and robustness**.

Table 16.1.2.1: Unit Function Table

Module	Function	Expected Output	Actual Output	Status
Authentication	validateUser()	Valid credentials return success	Returned success	Pass
Lawyer Management	addLawyerProfile ()	Lawyer profile stored in database	Stored successfully	Pass
Appointment System	createAppointmen t()	Appointment record created	Created successfully	Pass

Payment Module	processPayment()	Payment recorded and confirmed	Recorded successfully	Pass
Feedback Module	submitFeedback()	Feedback saved and retrievable	Saved successfully	Pass

Table 6.1.2.1 provides a breakdown of the unit testing phase. Each row represents a specific function or method tested within the system. The “Expected Output” column defines what the function was intended to return, while the “Actual Output” shows the observed behavior during testing.

Since all outputs matched their expectations, the unit tests were marked as successful. This validation ensured that each isolated component worked correctly before integration with other modules, reducing the likelihood of system-level failures and ensuring the Lawyer Management System performs reliably in real-world scenarios.

6.1.3 Functional Testing

Functional testing ensured that all use cases described in the UML diagram for the Lawyer Management System worked as intended, meeting both client, lawyer, and admin requirements.

Table 6.1.3.1: Test Case Table

Test Case ID	Use Case	Input	Expected Output	Actual Output	Status
TC-01	Register / Login	Valid credentials	Redirect to dashboard	Successful login	Pass
TC-02	Search & Filter Lawyers	Filter by specialization	Filtered results displayed	As expected	Pass
TC-03	View Lawyer Profile	Select lawyer	Display profile details	As expected	Pass
TC-04	Book Appointment	Choose date & submit	Appointment confirmed	As expected	Pass
TC-05	Manage Lawyer Profiles	Admin edits listing	Updated successfully	As expected	Pass

Table 6.1.3.1: presents the functional test cases for the major operations of the system. Each “Test Case ID” corresponds to a specific client, lawyer, or admin function derived from the UML use case diagram. The “Input” column specifies the action performed or data entered by the tester, while “Expected Output” and “Actual Output” show the intended and observed system behavior, respectively.

The consistent “Pass” status across all test cases confirms that the Lawyer Management System fulfills its intended functional requirements, offering a reliable and smooth workflow for all users.

6.1.4 Integration Testing

Integration testing was conducted to ensure smooth data flow and proper interaction between interconnected modules, such as the **AI system, lawyer management, appointment scheduling, payment processing, and dashboard analytics.**

Tested Integrations:

Client ↔ Lawyer Search & AI Recommendation: Ensures that AI-based lawyer suggestions are dynamically updated based on user preferences and interactions.

Admin ↔ Lawyer Management Module ↔ Database: Verifies that new lawyer profiles or updates appear instantly in the system.

Appointment ↔ Dashboard ↔ Notification System: Confirms that booked appointments are synchronized and notifications are delivered to clients, lawyers, and admins.

Payment ↔ Booking ↔ Dashboard: Ensures that payments are correctly linked to appointments and reflected in the admin dashboard.

Table 6.1.4.1: Integration Testing Table

Integration ID	Modules Integrated	Expected Result	Actual Result	Status
IN-01	Client & AI System	Accurate AI-based lawyer suggestions	As expected	Pass
IN-02	Admin & Lawyer Module	New or updated profiles appear instantly	As expected	Pass
IN-03	Appointment & Dashboard	Synced appointment updates	As expected	Pass

Table 6.1.4.1: documents the results of integration testing. Each row represents a set of modules or subsystems tested together. The “Expected Result” defines how the system should behave after integration, while the “Actual Result” confirms its real-world performance.

The consistent “Pass” status demonstrates that the communication and data flow between all major components—such as the Client Module, AI Recommendation System, Appointment Module, Payment Module, and Admin Dashboard—were efficient, error-free, and stable.

6.2 Tools and Technologies

During the testing and evaluation phases, a range of tools, libraries, and frameworks were utilized to ensure that the **Lawyer Management System** performed efficiently, securely, and accurately. Each tool served a specialized purpose—whether in development, testing, UI design, API validation, or AI-based processing. This combination of technologies ensured reliable performance, smooth workflows, and consistent user experience across all client, lawyer, and admin modules.

Table 6.2.1: Tools and Technologies Table

Tool / Framework	Category	Purpose / Description
React.js	Frontend Framework	Used for building an intuitive, responsive, and user-friendly interface for clients, lawyers, and administrators.
Node.js	Backend Runtime Environment	Handles server-side logic, API requests, authentication, and communication between frontend and database.

Express.js	Web Framework	Provides routing, middleware, and REST API handling for managing lawyer profiles, appointments, payments, and user data.
MongoDB Atlas	Database	Securely stores client records, lawyer profiles, appointment schedules, payment logs, and user feedback.
Python / ML Models (optional)	AI & Data Processing	Used for generating AI-driven lawyer recommendations based on specialization, case type, and user preferences.
Postman	API Testing Tool	Used for validating all backend API endpoints such as login, lawyer search, appointment booking, and payment verification.
Jest / Manual Testing	Testing Framework	Supports unit testing, functional testing, and integration testing to ensure accuracy and reliability of all modules.
Figma	UI/UX Design Tool	Used to design interface mockups and ensure consistent layout, interactions, and visual flow across the platform.
Git & GitHub	Version Control	
Visual Studio Code (VS Code)	Development Environment	The primary editor for writing, debugging, and managing the full-stack development of the Lawyer Management System.

Table 6.2.1: outlines all tools and technologies used during the development and testing of the Lawyer Management System. Each tool is associated with its specific purpose, showing how different components of the technology stack worked together to create a stable, secure, and feature-rich platform. This modern tech stack ensured effective testing, consistent development workflows, and smooth deployment.

6.3 Summary

The testing and evaluation phase successfully validated the **Law Sphere** accuracy, usability, and operational reliability. Through a structured approach involving manual testing, unit testing, functional testing, and integration testing, every major module was assessed against the system's requirements and performance standards.

All test cases passed successfully, and no major functional defects were detected during evaluation. Minor UI inconsistencies identified during manual testing were quickly resolved, ensuring seamless navigation and user interaction across the platform.

The integration between modules—especially the interaction between the AI recommendation system, appointment scheduling, payment processing, and admin dashboard—was smooth and error-free. The stable flow of data between the backend APIs and the MongoDB Atlas database confirmed the robustness of the system architecture.

Overall, the testing phase concluded that the Lawyer Management System is stable, scalable, and fully ready for deployment. It fulfills all project objectives by providing secure authentication, efficient lawyer-client interaction, AI-powered recommendations, and a highly intuitive experience for clients, lawyers, and administrators.

.

Chapter 7

CONCLUSION AND FUTURE WORK

7 Conclusion

The development of the **Law Sphere** successfully achieved its primary objective of creating a modern, intelligent, and user-friendly platform for managing legal cases, client appointments, documentation, and lawyer–client interactions. The system effectively bridges the gap between lawyers, clients, and administrators by integrating automation, digital record-keeping, and smart decision-support features into one unified ecosystem.

Throughout the project, core emphasis was placed on **usability, functionality, security, and scalability**. A modular system architecture was designed to ensure that each component—such as client management, case tracking, document uploading, appointment scheduling, payment handling, and admin control—operates independently while integrating seamlessly with the rest of the platform. This approach made the system easier to develop, test, maintain, and expand.

The user interface, designed for both lawyers and clients, offers a clean and intuitive experience, while the backend ensures secure and efficient handling of sensitive legal data. Reliable data storage, authentication mechanisms, and role-based access were integrated to ensure confidentiality and prevent unauthorized access to client information. Additionally, automated reminders, smart notifications, and case-progress tracking improved efficiency for both lawyers and clients.

Comprehensive testing—including unit, manual, functional, and integration testing—validated the stability and correctness of each module. All system components performed in accordance with the defined requirements, and all test cases were successfully passed. The seamless integration between case management, client modules, appointment scheduling, and the admin dashboard demonstrated strong backend logic and smooth data flow.

In conclusion, the Lawyer Management System demonstrates how modern web technologies can transform traditional legal processes by enabling digital case management, reducing manual workload, and improving client–lawyer communication. The system fulfills all planned objectives and establishes a reliable foundation for future extensions in the legal tech domain.

7.1 Future Work

Although the current version of the Lawyer Management System meets its intended objectives, there are numerous opportunities to expand and enhance the platform. The following areas are recommended for future development:

7.1.1 Mobile Application Development

Create dedicated Android and iOS mobile applications to provide easier access for lawyers and clients, enabling them to manage cases and appointments on the go.

7.1.2 AI-Based Legal Assistance and Automation

Integrate advanced AI models to analyze case history, suggest legal strategies, auto-generate legal documents, or provide intelligent recommendations based on precedents.

7.1.3 Real-Time Chat and Virtual Legal Assistant

Add a secure messaging system or an AI-driven legal assistant to support clients in asking questions, receiving updates, or communicating with lawyers instantly.

7.1.4 Secure Online Payment Integration

Introduce payment gateways for clients to pay consultation fees, case charges, or document-processing fees directly from the platform.

7.1.5 Advanced Data Analytics Dashboard

Enhance the admin dashboard with analytics such as lawyer performance, appointment trends, revenue statistics, case timelines, and predictive insights for workload distribution.

7.1.6 Cloud Deployment and Scalability

Deploy the system on cloud platforms (AWS, Azure, or Google Cloud) to support larger datasets, multi-branch law firms, and high availability across different geographical regions.

7.1.7 Enhanced Security and Compliance

Implement advanced security features such as Two-Factor Authentication (2FA), complete data encryption, secure document handling, and compliance with legal data privacy standards.

7.1.8 Integration With Court Systems & Legal Databases

Integrate APIs for accessing legal databases, court schedules, or cause lists to automate updates related to hearing dates, case status, or legal notices.

Chapter 8

REFERENCE

8 References

- [1] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. Sebastopol, CA, USA: O'Reilly Media, 2020. [Online]. Available: <https://www.oreilly.com/library/view/javascript-the-definitive/9781491952023/>
- [2] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Apps*. Sebastopol, CA, USA: O'Reilly Media, 2017. [Online]. Available: <https://www.oreilly.com/library/view/learning-react-2nd/9781492051718/>
- [3] T. Holowaychuk, *Express.js Guide: The Comprehensive Book on Express.js*. Leanpub, 2019. [Online]. Available: <https://leanpub.com/expressjs>
- [4] MongoDB Inc., “MongoDB Documentation,” 2024. [Online]. Available: <https://www.mongodb.com/docs/>
- [5] Node.js Foundation, “Node.js Documentation,” 2024. [Online]. Available: <https://nodejs.org/en/docs>
- [6] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022. [Online]. Available: <https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/>
- [7] Postman Inc., “Postman API Platform Documentation,” 2024. [Online]. Available: <https://learning.postman.com/>
- [8] GitHub, “GitHub Documentation,” 2024. [Online]. Available: <https://docs.github.com/>
- [9] Jest Community, “Jest Testing Framework Documentation,” 2024. [Online]. Available: <https://jestjs.io/docs>
- [10] Figma, “Figma Design System Documentation,” 2024. [Online]. Available: <https://help.figma.com/>
- [11] OpenAI, “OpenAI API Documentation,” 2024. [Online]. Available: <https://platform.openai.com/docs>
- [12] Google Cloud, “Google Cloud Documentation,” 2024. [Online]. Available: <https://cloud.google.com/docs>
- [13] World Wide Web Consortium (W3C), “HTML & CSS Standards,” 2024. [Online]. Available: <https://www.w3.org/TR/>
- [14] UML.org, “Unified Modeling Language (UML) Specifications,” 2024. [Online]. Available: <https://www.uml.org/>
- [15] Stack Overflow, “Developer Documentation and Technical Discussions,” 2024. [Online]. Available: <https://stackoverflow.com/>